

Universidade de Brasília – UnB

Faculdade de Ciências e Tecnologias em Engenharia – FCTE

Engenharia de Software

Correção de vulnerabilidades em código com LLMs: uma avaliação da integração com ferramentas SAST, testes automatizados, estratégias de contextualização e raciocínio guiado

Autor: Matheus de Siqueira Brant e Caio Magalhães Lamego

Orientador: Profa. Dra. Elaine Venson

Brasília, DF

2026



Matheus de Siqueira Brant e Caio Magalhães Lamego

Correção de vulnerabilidades em código com LLMs: uma avaliação da integração com ferramentas SAST, testes automatizados, estratégias de contextualização e raciocínio guiado

Monografia submetida ao curso de graduação em Engenharia de Software da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de Software.

Universidade de Brasília – UnB

Faculdade de Ciências e Tecnologias em Engenharia – FCTE

Orientador: Profa. Dra. Elaine Venson

Brasília, DF

2026

Matheus de Siqueira Brant e Caio Magalhães Lamego
Correção de vulnerabilidades em código com LLMs: uma avaliação da integração com ferramentas SAST, testes automatizados, estratégias de contextualização e raciocínio guiado/ Matheus de Siqueira Brant e Caio Magalhães Lamego. – Brasília, DF, 2026-
71 p. : il. ; 30 cm.

Orientador: Profa. Dra. Elaine Venson

Trabalho de Conclusão de Curso – Universidade de Brasília – UnB
Faculdade de Ciências e Tecnologias em Engenharia – FCTE , 2026.

1. segurança de software. 2. engenharia de software. I. Profa. Dra. Elaine Venson. II. Universidade de Brasília. III. Faculdade de Ciências e Tecnologias em Engenharia. IV. Correção de vulnerabilidades em código com LLMs: uma avaliação da integração com ferramentas SAST, testes automatizados, estratégias de contextualização e raciocínio guiado

CDU 02:141:005.6

Matheus de Siqueira Brant e Caio Magalhães Lamego

Correção de vulnerabilidades em código com LLMs: uma avaliação da integração com ferramentas SAST, testes automatizados, estratégias de contextualização e raciocínio guiado

Monografia submetida ao curso de graduação em Engenharia de Software da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em Engenharia de Software.

Trabalho aprovado. Brasília, DF, 16 de julho de 2026:

Profa. Dra. Elaine Venson
Orientador

Dra. Cristiane Soares Ramos
Convidado 1

Indiana Medeiros
Convidado 2

Brasília, DF
2026

Resumo

Modelos de Linguagem de Grande Escala (LLMs) têm sido cada vez mais utilizados no desenvolvimento de software, inclusive em tarefas de geração, explicação e correção de código. No entanto, seu uso em segurança ainda exige cautela, pois correções geradas automaticamente podem ser incompletas, inseguras ou comprometer o funcionamento original da aplicação. Este trabalho investiga a integração entre LLMs, ferramentas de Análise Estática de Segurança (SAST), testes automatizados e estratégias de contextualização para apoiar a correção de *issues* de segurança em código-fonte. A metodologia proposta prevê a aplicação de um *pipeline* automatizado em um projeto *open source*, utilizando as vulnerabilidades efetivamente detectadas pela ferramenta SAST. Como etapa inicial, foi desenvolvida uma prova de conceito para validar o fluxo de identificação, geração de correções, reanálise estática, execução de testes e consolidação de métricas. O trabalho estabelece, assim, as bases técnicas e metodológicas para avaliar a eficácia do *pipeline* no TCC 2.

Palavras-chave: modelos de linguagem de grande escala; análise estática de segurança; correção de vulnerabilidades; engenharia de prompt; RAG; *pipeline*.

Abstract

Large Language Models (LLMs) have been increasingly used in software development, including tasks such as code generation, explanation, and correction. However, their use in software security still requires caution, since automatically generated fixes may be incomplete, insecure, or may compromise the original behavior of the application. This work investigates the integration of LLMs, Static Application Security Testing (SAST) tools, automated tests, and contextualization strategies to support the correction of security issues in source code. The proposed methodology applies an automated pipeline to an open source project, using the vulnerabilities effectively detected by the SAST tool. As an initial step, a proof of concept was developed to validate the workflow for issue identification, fix generation, static reanalysis, test execution, and metric consolidation. Thus, this work establishes the technical and methodological foundations to evaluate the effectiveness of the pipeline in TCC 2.

Keywords: large language models; static application security testing; vulnerability repair; prompt engineering; RAG; pipeline.

Lista de ilustrações

Figura 1 – Visão geral do fluxo metodológico adotado na pesquisa. 34
Figura 2 – Fluxo de correção de uma única vulnerabilidade, executado em paralelo para diferentes vulnerabilidades e com tentativas progressivas por estratégias de prompt. 42
Figura 3 – Estrutura do identificador utilizado na nomenclatura dos artefatos gerados pelo *pipeline*. 45
Figura 4 – Primeira parte da execução completa do *pipeline* da PoC. 62
Figura 5 – Segunda parte da execução completa do *pipeline* da PoC. 63

Lista de tabelas

Tabela 1 – Cronograma previsto para o desenvolvimento do TCC 2 49
Tabela 2 – Resultados operacionais das execuções da PoC 64
Tabela 3 – Métricas e classificação final das correções 64
Tabela 4 – Métricas gerais obtidas na execução da PoC 64
Tabela 5 – Métricas consolidadas por estratégia de *prompt* 65
Tabela 6 – Métricas consolidadas por caso vulnerável 65

Lista de abreviaturas e siglas

ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface
CoT	Chain-of-Thought
CVE	Common Vulnerabilities and Exposures
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
IA	Inteligência Artificial
IDOR	Insecure Direct Object Reference
LLM	Large Language Model
NVD	National Vulnerability Database
OWASP	Open Worldwide Application Security Project
PoC	Proof of Concept
RAG	Retrieval-Augmented Generation
SAST	Static Application Security Testing
SSRF	Server-Side Request Forgery
XSS	Cross-Site Scripting

Lista de símbolos

$\sum$	Operador de somatório
R_j	Resultado da remoção da *issue* real j após a correção
F_j	Resultado da preservação funcional da correção aplicada à *issue* real j
M_j	Indicador de não introdução de novas *issues* após a correção da *issue* real j
m	Número total de tentativas de correção realizadas no projeto real
I_{antes}	Número de *issues* de segurança detectadas antes da aplicação das correções aceitas
I_{depois}	Número de *issues* de segurança detectadas após a aplicação das correções aceitas

Sumário

1 INTRODUÇÃO 12
1.1 Contexto 12
1.2 Problema 13
1.3 Objetivos 15
1.3.1 Objetivo Geral 15
1.3.2 Objetivos Específicos 15
1.4 Metodologia 15
1.5 Organização do Trabalho 16

2 REFERENCIAL TEÓRICO 17
2.1 Modelos de Linguagem de Grande Escala (LLMs) 17
2.2 Engenharia de Prompt 20
2.3 Vulnerabilidades de Software e OWASP 10 25
2.4 Ferramentas de Análise Estática de Segurança (SAST) 28
2.5 Trabalhos Relacionados 31

3 METODOLOGIA 33
3.1 Plano Metodológico 33
3.2 Contexto de aplicação do experimento 34
3.3 Objeto de estudo: OWASP Juice Shop 35
3.4 Seleção das vulnerabilidades de segurança 36
3.5 Ferramenta de análise estática 37
3.6 Estratégias de prompt 38
3.7 Estratégia de RAG simplificado 39
3.8 Pipeline 41
3.9 Métricas de avaliação 43
3.10 Armazenamento e organização dos dados 44
3.11 Análise dos dados 46
3.12 Limitações do estudo 47
3.13 Declaração de Uso de Inteligência Artificial 48
3.14 Cronograma 48

4 PROVA DE CONCEITO 50
4.1 Visão geral 50
4.2 Elaboração da PoC 50
4.2.1 Escolha e fundamentação das vulnerabilidades 51

4.2.2 Criação dos códigos vulneráveis 52
4.2.3 Elaboração dos testes automatizados 53
4.2.4 Elaboração das estratégias de prompt 54
4.2.5 Definição do fluxo experimental 55
4.3 Estrutura do projeto 56
4.4 Fluxo de execução e armazenamento dos resultados 58
4.5 Métricas de avaliação 60
4.6 Execução da PoC 61
4.7 Interpretação dos resultados 61
4.8 Contribuição da PoC 66

5 CONSIDERAÇÕES FINAIS 67

REFERÊNCIAS 68

1 Introdução

1.1 Contexto

Nos últimos anos, especialmente a partir de 2022, os Modelos de Linguagem de Grande Escala, do inglês *Large Language Models* (LLMs), passaram a integrar de forma cada vez mais significativa o processo de desenvolvimento de software. Essas ferramentas têm sido utilizadas em diversas atividades, como geração de código, explicação de trechos, sugestão de correções e apoio à análise de programas. Com isso, os LLMs passaram a ocupar um papel relevante no cotidiano de desenvolvedores e equipes de engenharia de software, ainda que continuem apresentando limitações em tarefas que demandam maior precisão e compreensão da lógica do código (ZHOU et al., 2024).

Com esse crescimento, também emergiram novas formas de desenvolvimento de software apoiadas por IA. Uma delas é o chamado *vibe coding*, no qual o desenvolvedor descreve, em linguagem natural, o que deseja construir e realiza ajustes sobre o código gerado pelo modelo. Paralelamente, ganhou destaque uma abordagem mais controlada, conhecida como *AI-assisted engineering*, na qual a IA atua como suporte dentro de um processo supervisionado, enquanto o desenvolvedor permanece responsável por revisar, validar e decidir o que deve ser incorporado ao sistema (OSMANI, 2025). Essa distinção é relevante, pois evidencia que o impacto da IA no desenvolvimento de software não depende apenas das capacidades do modelo, mas também da forma como ele é integrado ao processo de desenvolvimento.

Apesar dos ganhos de produtividade proporcionados por essas ferramentas, o uso de LLMs em tarefas relacionadas à segurança de software tem gerado preocupações. Estudos indicam que assistentes de programação baseados em IA podem produzir código inseguro em uma parcela significativa dos casos. Em uma pesquisa sobre o GitHub Copilot, por exemplo, cerca de 40% dos programas gerados nos cenários analisados apresentavam vulnerabilidades, evidenciando que essas ferramentas podem acelerar o desenvolvimento, mas não devem ser utilizadas sem validação, especialmente em contextos críticos (PEARCE et al., 2021).

Outro fator relevante é que o desempenho desses modelos depende fortemente do contexto fornecido. Em tarefas de detecção de vulnerabilidades, pesquisas mostram que a inclusão de informações adicionais pode melhorar a capacidade da IA de diferenciar falhas reais de situações apenas aparentemente suspeitas (IRANMANESH et al., 2025).

Diante disso, trabalhos recentes passaram a explorar abordagens em que a LLM não atua de forma isolada, mas em conjunto com ferramentas externas de apoio. Entre

essas ferramentas, destacam-se as soluções de *Static Application Security Testing* (SAST), que analisam o código-fonte sem executá-lo e apontam possíveis vulnerabilidades com base em padrões e regras conhecidas. Embora sejam úteis por ampliarem a cobertura da análise, essas ferramentas indicam condições potencialmente vulneráveis que ainda precisam ser interpretadas e validadas. Assim, seus alertas não devem ser tratados isoladamente como correções ou comprovações definitivas de vulnerabilidade, mas como evidências técnicas que podem orientar etapas posteriores de análise e remediação (IRANMANESH et al., 2025).

Nesse contexto, a integração entre LLMs e ferramentas SAST apresenta-se como uma alternativa promissora para tornar a correção de vulnerabilidades mais eficiente e adequada ao contexto do código analisado. Enquanto as ferramentas SAST contribuem para a identificação de possíveis problemas no código-fonte, os LLMs podem apoiar uma análise mais contextualizada, auxiliando na compreensão e na correção das vulnerabilidades detectadas. Estudos recentes indicam que abordagens iterativas, baseadas no uso de contexto adicional e no feedback de ferramentas externas, tendem a produzir resultados mais confiáveis do que interações isoladas fundamentadas em um único prompt (SRIRAM et al., 2026). É a partir dessa possibilidade de combinação entre essas tecnologias que se estabelece a motivação deste trabalho.

1.2 Problema

Embora a integração entre ferramentas de segurança e inteligência artificial se mostre promissora, sua aplicação prática ainda apresenta diversos desafios. Um dos principais está relacionado ao fato de que os modelos de linguagem, quando utilizados de forma isolada para corrigir vulnerabilidades, ainda não produzem resultados suficientemente confiáveis. Estudos recentes indicam que esses modelos conseguem identificar a existência de falhas com desempenho razoável, mas ainda apresentam grande dificuldade para gerar correções efetivas. Em um desses trabalhos, por exemplo, o modelo alcançou 76% de acerto na detecção de vulnerabilidades, mas apenas 15% de sucesso no reparo quando não recebeu contexto adicional sobre o problema (KHAN; LIU; GAO, 2025).

Além da baixa taxa de sucesso no reparo, outro problema relevante é que, em muitos casos, a tentativa de corrigir a vulnerabilidade pode comprometer o funcionamento original do software. Em vez de produzir uma solução segura e funcional, o modelo pode remover partes importantes da lógica da aplicação ou gerar um código que, embora elimine a falha de segurança, prejudique a utilidade do sistema. Esse comportamento evidencia que corrigir uma vulnerabilidade não consiste apenas em remover o erro, mas também em preservar o comportamento esperado do programa (DAI; XU; TAO, 2025).

Por outro lado, depender exclusivamente de ferramentas tradicionais de análise

estática também não resolve o problema. Embora essas ferramentas sejam importantes para localizar possíveis falhas no código, seus alertas ainda precisam ser interpretados, priorizados e validados por desenvolvedores. Além disso, a identificação de uma *issue* não implica, por si só, a existência de uma correção segura e funcional. Dessa forma, o SAST contribui para apontar evidências técnicas sobre possíveis vulnerabilidades, mas não substitui etapas posteriores de análise, remediação e validação (IRANMANESH et al., 2025).

Pesquisas recentes indicam que o desempenho das LLMs em tarefas de segurança pode melhorar quando o modelo recebe mais contexto e quando seu processo de resposta é conduzido de forma mais estruturada (TAO et al., 2025). Nesse cenário, estratégias como *Retrieval-Augmented Generation* (RAG), que permite recuperar informações relevantes antes da geração da resposta, e *Chain-of-Thought* (CoT), que incentiva um raciocínio em etapas, surgem como alternativas promissoras para apoiar tanto a interpretação dos alertas quanto o reparo das vulnerabilidades. Ainda assim, não está claro até que ponto essas estratégias realmente aumentam a eficácia da LLM quando ela atua em conjunto com ferramentas SAST (SRIRAM et al., 2026).

Assim, o problema central desta pesquisa surge justamente das limitações dessas abordagens quando utilizadas de forma isolada. De um lado, as LLMs ainda apresentam dificuldade para reparar vulnerabilidades de maneira confiável e sem comprometer a funcionalidade do software. De outro, ferramentas SAST são capazes de apontar possíveis falhas no código, mas não produzem, por si só, uma correção segura e funcional. Essa situação evidencia uma lacuna importante na engenharia de software atual: compreender até que ponto essas tecnologias podem atuar de forma complementar, com o SAST auxiliando na localização e descrição dos problemas, e a LLM contribuindo para gerar correções que possam ser posteriormente validadas por reanálise estática e testes automatizados.

Com base nisso, esta pesquisa busca responder às seguintes questões:

- Q1: É possível gerar correções válidas para vulnerabilidades por meio de um *pipeline* automatizado que integra LLMs, ferramentas SAST e testes automatizados, considerando a remoção dos alertas, a preservação dos testes automatizados e a não introdução de novas *issues* de segurança?
- Q2: Como diferentes estratégias de prompt, incluindo prompt simples, raciocínio guiado e enriquecimento de contexto por RAG simplificado, influenciam a taxa de correções válidas geradas pelo modelo de linguagem?
- Q3: O desempenho do *pipeline* varia de acordo com o tipo de vulnerabilidade ou categoria associada ao OWASP Top 10?

- Q4: Quais limitações práticas surgem ao aplicar um *pipeline* automatizado de correção de vulnerabilidades em uma base de código real, com dependências, arquitetura própria e suíte de testes existente?

1.3 Objetivos

1.3.1 Objetivo Geral

Investigar a eficácia de um fluxo iterativo que integra Modelos de Linguagem de Grande Escala (LLMs), ferramentas de Análise Estática de Segurança (SAST) e testes automatizados na correção de vulnerabilidades em código-fonte, considerando a remoção dos alertas de segurança, a preservação da funcionalidade original e a não introdução de novas *issues*.

1.3.2 Objetivos Específicos

- Desenvolver um *pipeline* automatizado que integre uma ferramenta de análise estática de segurança, um modelo de linguagem e testes automatizados para apoiar a correção de *issues* de segurança em código-fonte.
- Definir e aplicar diferentes estratégias de *prompt*, incluindo *prompt* simples, raciocínio guiado e enriquecimento de contexto por RAG simplificado, a fim de comparar seu impacto na geração de correções válidas.
- Avaliar as correções geradas a partir da reanálise SAST, da execução dos testes automatizados e da verificação de que novas *issues* de segurança não foram introduzidas.
- Analisar os resultados obtidos no projeto, considerando a taxa de remediação validada, os tipos de *issues* detectadas, as falhas de correção e as limitações práticas do *pipeline*.

1.4 Metodologia

Este trabalho propõe uma investigação empírica e experimental sobre a eficácia de um fluxo iterativo que integra Modelos de Linguagem de Grande Escala (LLMs) e ferramentas de Análise Estática de Segurança (SAST) na correção automatizada de vulnerabilidades. A pesquisa parte da ideia de que, embora LLMs possam auxiliar na identificação e no reparo de falhas, seu desempenho tende a melhorar quando elas recebem contexto adicional e operam em conjunto com ferramentas externas de apoio.

No TCC 1, essa proposta é inicialmente validada por meio de uma prova de conceito em ambiente controlado, composta por casos vulneráveis representativos, testes automatizados e execução integrada do *pipeline*. Essa etapa tem como finalidade verificar a viabilidade técnica do fluxo proposto antes de sua aplicação em uma base de código real. No TCC 2, pretende-se evoluir essa abordagem para um projeto público, considerando as vulnerabilidades efetivamente identificadas pela ferramenta SAST selecionada.

Inicialmente, será selecionado um projeto público que possa ser instalado e executado em ambiente local, possua estrutura modular e apresente *issues* de segurança detectáveis por uma ferramenta SAST compatível com a linguagem. As vulnerabilidades analisadas serão determinadas a partir dos alertas emitidos pela ferramenta SAST sobre o código-fonte do projeto, respeitando critérios de elegibilidade relacionados à possibilidade de reprodução, correção e validação automatizada.

Em seguida, os alertas gerados pela análise estática serão utilizados como entrada para o *pipeline*. Para cada *issue*, serão aplicadas diferentes estratégias de *prompt*, incluindo instrução simples, raciocínio guiado, RAG simplificado e a combinação entre RAG simplificado e raciocínio guiado. O objetivo é verificar se o aumento de contexto e estruturação da tarefa melhora a qualidade das correções propostas pela LLM.

Após a geração das correções, as modificações serão aplicadas em cópias isoladas do projeto e avaliadas por meio de nova execução da ferramenta SAST e da suíte de testes automatizados disponível. Uma correção será considerada válida apenas quando a *issue* original deixar de ser detectada, os testes continuarem passando e nenhuma nova *issue* de segurança for introduzida. Os resultados serão consolidados em arquivos estruturados para permitir a comparação entre estratégias de *prompt*, tipos de *issues* detectadas e limitações observadas durante a aplicação em um projeto.

Dessa forma, a metodologia pretende medir quantitativamente o desempenho da arquitetura proposta, como também compreender em quais condições a combinação entre análise estática, contexto adicional e raciocínio orientado por LLMs se mostra mais eficaz para apoiar tarefas de segurança de software.

1.5 Organização do Trabalho

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta o referencial teórico que fundamenta a pesquisa; o Capítulo 3 descreve a metodologia proposta para a continuidade do estudo, incluindo o planejamento da coleta e análise dos dados no TCC 2; o Capítulo 4 apresenta a prova de conceito desenvolvida no TCC 1, destacando sua estrutura, execução, métricas e resultados obtidos; por fim, o Capítulo 5 apresenta as considerações finais desta etapa do trabalho.

2 Referencial Teórico

Este capítulo apresenta a fundamentação teórica da pesquisa, abordando os Modelos de Linguagem de Grande Escala (LLMs), suas limitações e aplicações no desenvolvimento de software. Também são discutidos os conceitos de vulnerabilidades de código e o uso do *OWASP Top 10* como referência para avaliação de segurança. Em seguida, são apresentados os princípios da Engenharia de Prompt, com destaque para técnicas como *Chain-of-Thought (CoT)* e Geração Aumentada por Recuperação (RAG), além do papel das ferramentas de Análise Estática de Segurança (SAST) no diagnóstico de falhas. Por fim, são discutidos os trabalhos relacionados que fundamentam a proposta de integração iterativa desenvolvida neste estudo.

2.1 Modelos de Linguagem de Grande Escala (LLMs)

O avanço dos Modelos de Linguagem de Grande Escala, do inglês *Large Language Models* (LLMs), está diretamente associado à consolidação da arquitetura Transformer, proposta por (VASWANI et al., 2023). Diferentemente das redes neurais recorrentes, que processam sequências de forma essencialmente ordenada e dependente de estados anteriores, o *Transformer* utiliza mecanismos de autoatenção, para relacionar diferentes posições de uma sequência de entrada simultaneamente, permitindo que o modelo avalie a relevância de cada token em relação aos demais. Em tarefas envolvendo código-fonte, essa capacidade é especialmente importante, pois uma vulnerabilidade pode depender de relações entre variáveis, chamadas de função, fluxos de dados e decisões de controle distribuídas ao longo de diferentes trechos do programa.

A arquitetura *Transformer* é composta, em sua forma original, por blocos codificadores e decodificadores. O codificador transforma a sequência de entrada em representações contextuais, enquanto o decodificador gera uma sequência de saída de maneira autorregressiva, isto é, prevendo cada novo token com base nos tokens já gerados. A partir dessa estrutura, os modelos de linguagem passaram a ser organizados em três famílias principais: modelos apenas codificadores, modelos apenas decodificadores e modelos codificador-decodificador. Essa diferenciação é relevante para a Engenharia de Software porque cada arquitetura apresenta maior adequação a tipos distintos de tarefas, como compreensão de código, detecção de vulnerabilidades, geração de programas e reparo automático (VASWANI et al., 2023).

Antes de serem utilizados como assistentes de programação, os LLMs passam por um ciclo de treinamento composto por diferentes etapas. Inicialmente, grandes volumes de dados são coletados, limpos, deduplicados e transformados em tokens. Em seguida, ocorre

o pré-treinamento, geralmente de forma auto-supervisionada, no qual o modelo aprende regularidades estatísticas da linguagem natural e, no caso dos modelos especializados em programação, também de código-fonte (MINAEE et al., 2025).

O treinamento em grandes bases de código explica por que esses modelos conseguem gerar programas sintaticamente plausíveis, completar funções, explicar trechos e sugerir correções. No entanto, esse aprendizado não equivale a uma compreensão formal da semântica do programa. O modelo aprende padrões recorrentes presentes nos dados de treinamento e os utiliza para prever a continuação mais provável de uma sequência (MINAEE et al., 2025). Assim, quando solicitado a corrigir uma vulnerabilidade, o LLM não executa, necessariamente, uma prova de segurança nem garante que a lógica de negócio será preservada. Ele produz uma resposta provável com base no contexto recebido, o que torna essencial a presença de validação externa, especialmente em tarefas críticas (SRIRAM et al., 2026).

Com o aumento de escala desses modelos, a literatura passou a observar habilidades chamadas emergentes, isto é, capacidades que se tornam mais evidentes em modelos maiores e mais bem treinados. Entre elas, destacam-se o seguimento de instruções, o raciocínio em múltiplas etapas e o aprendizado em contexto. O aprendizado em contexto permite que o modelo adapte sua resposta a uma tarefa específica apenas a partir das instruções, exemplos e informações fornecidas no prompt, sem atualizar seus pesos internos (MINAEE et al., 2025). Essa característica é fundamental para este trabalho, pois técnicas como *Chain-of-Thought* (CoT) e *Retrieval-Augmented Generation* (RAG) exploram justamente a capacidade do modelo de usar contexto adicional durante a inferência. No caso do reparo de vulnerabilidades, isso significa que o modelo pode receber o alerta emitido por uma ferramenta SAST, trechos de código relacionados, exemplos de correções anteriores, regras de segurança e documentação técnica para orientar sua resposta.

No contexto de código-fonte, os modelos apenas codificadores, ou *encoder-only*, são voltados principalmente para tarefas de compreensão. Como suas camadas de atenção conseguem acessar a sequência de entrada de forma bidirecional, esses modelos são adequados para analisar o código como um todo e produzir uma representação contextual do trecho analisado (JIANG et al., 2023). Exemplos dessa família incluem CodeBERT e GraphCodeBERT, que são frequentemente aplicados em tarefas de detecção de vulnerabilidades, classificação de código vulnerável e identificação de padrões suspeitos. Entretanto, por não serem projetados originalmente para geração autorregressiva de longas sequências, eles tendem a ser menos adequados para produzir diretamente uma correção completa (ZHOU et al., 2024).

Os modelos apenas decodificadores, ou *decoder-only*, seguem outra lógica. Eles são autorregressivos e foram projetados para prever o próximo token com base no contexto anterior. Essa característica os torna especialmente eficientes em tarefas de geração

contínua, como completar funções, escrever código a partir de descrições em linguagem natural e refatorar trechos. No contexto de desenvolvimento de software, esses modelos como GPT, o Codex, o Code Llama se popularizaram justamente por sua capacidade de produzir código funcional a partir de prompts curtos. No entanto, essa mesma natureza generativa também traz riscos: o modelo pode gerar trechos que parecem corretos, mas contêm vulnerabilidades ou não preservam requisitos implícitos do sistema (MINAEE et al., 2025).

Já os modelos codificador-decodificador, ou *encoder-decoder*, combinam as duas capacidades anteriores. O codificador lê e representa a entrada, enquanto o decodificador gera uma saída condicionada a essa representação. Por isso, essa arquitetura é especialmente adequada para tarefas de transformação de sequência, nas quais uma entrada deve ser convertida em uma saída relacionada. No campo de código, exemplos importantes incluem CodeT5 e PLBART. Esses modelos podem ser utilizados tanto para compreensão quanto para geração, mas se destacam em tarefas como tradução entre linguagens, sumarização de código, refinamento e reparo de defeitos (MINAEE et al., 2025).

A literatura sobre detecção e reparo de vulnerabilidades confirma essa divisão funcional entre arquiteturas. Estudos de revisão indicam que modelos *encoder-only* aparecem com frequência em tarefas de detecção de vulnerabilidades, enquanto modelos *decoder-only* e *encoder-decoder* são mais recorrentes em tarefas de reparo (ZHOU et al., 2024). Essa diferença ocorre porque detectar uma vulnerabilidade exige principalmente compreender, representar e classificar um trecho de código, enquanto reparar exige gerar uma nova versão do programa. Embora os modelos *encoder-decoder* pareçam, em princípio, a opção mais completa por combinarem leitura contextual da entrada e geração da saída, eles não são necessariamente a melhor escolha em todos os cenários. Seu desempenho depende de fatores como escala do modelo, dados de pré-treinamento, tipo de ajuste fino, tamanho da janela de contexto e alinhamento com instruções. Além disso, quando tratados como uma arquitetura única para múltiplas tarefas, esses modelos podem apresentar desempenho abaixo do esperado em algumas aplicações específicas. Em tarefas complexas de reparo de vulnerabilidades, modelos *decoder-only* de maior escala e mais bem alinhados para seguir instruções podem superar modelos *encoder-decoder* menores, especialmente quando recebem contexto adicional, exemplos, alertas de ferramentas SAST e feedback iterativo. Portanto, a escolha da arquitetura influencia diretamente o desenho de um *pipeline* de segurança: modelos *encoder-only* tendem a ser mais adequados para triagem e classificação de alertas, modelos *encoder-decoder* são úteis para transformações localizadas de código, e modelos *decoder-only* comerciais ou especializados em código podem ser mais apropriados para a geração de correções em fluxos iterativos e contextualizados.

Apesar desses avanços, o uso de LLMs em tarefas de segurança ainda apresenta limitações relevantes. Um dos problemas centrais é que os modelos aprendem a partir de

grandes quantidades de código produzido por desenvolvedores reais, incluindo exemplos corretos, inseguros, desatualizados ou incompletos (CHEN et al., 2021). Assim, quando não recebem restrições explícitas, podem replicar padrões inseguros presentes nos dados de treinamento. Em vez de selecionar automaticamente a alternativa mais segura, o modelo pode escolher a solução mais comum ou mais provável nos dados em que foi treinado.

Um limite importante aparece quando o modelo é utilizado em uma abordagem *single-pass*, isto é, quando recebe um único prompt e gera uma resposta sem iteração, validação ou realimentação por ferramentas externas. Embora esse tipo de interação seja simples, ela é insuficiente para tarefas de segurança. O reparo de uma vulnerabilidade não consiste apenas em remover o trecho apontado como inseguro, mas em produzir uma modificação que elimine o risco, preserve a lógica original, mantenha a funcionalidade esperada e não introduza novas falhas. Estudos recentes mostram que técnicas de geração segura podem, em alguns casos, aumentar artificialmente a pontuação de segurança ao remover linhas vulneráveis ou produzir código residual sem relação com a tarefa original, fenômeno descrito como *garbage code* (DAI; XU; TAO, 2025). Nesses casos, o código pode parecer seguro para uma análise superficial, mas deixa de cumprir sua finalidade.

Dessa forma, a principal limitação dos LLMs em reparo de vulnerabilidades não está apenas na capacidade de gerar código, mas na ausência de garantias sobre segurança e funcionalidade quando o modelo atua isoladamente. Mesmo modelos avançados podem produzir respostas plausíveis, mas incorretas, inseguras ou incompletas. Por isso, trabalhos recentes defendem o uso de fluxos iterativos com validação externa que forneçam sinais objetivos para orientar a correção. (SRIRAM et al., 2026), por exemplo, propõem um fluxo com RAG e realimentação por múltiplas ferramentas, incluindo compilador, CodeQL e execução simbólica, reduzindo significativamente a taxa de vulnerabilidades nos programas avaliados. Esse tipo de resultado reforça a premissa deste trabalho: LLMs podem apoiar o reparo automático de vulnerabilidades, mas tendem a ser mais confiáveis quando integrados a ferramentas de análise, enriquecimento de contexto e mecanismos iterativos de validação.

2.2 Engenharia de Prompt

A engenharia de prompt consiste no processo sistemático de estruturar as entradas fornecidas a um Modelo de Linguagem de Grande Escala para induzir comportamentos adequados a uma determinada tarefa. Diferentemente do ajuste fino (*fine tuning*), no qual os parâmetros internos do modelo são modificados a partir de novos exemplos de treinamento, a engenharia de prompt procura explorar os conhecimentos e as capacidades já adquiridos pelo modelo durante seu pré-treinamento. Dessa forma, o modelo pode ser adaptado a diferentes atividades sem a necessidade de realizar um novo treinamento, o

que reduz os custos computacionais e permite a utilização de modelos comerciais cujos parâmetros internos não estão disponíveis (REYNOLDS; MCDONELL, 2021).

Um prompt não precisa ser composto apenas por uma pergunta ou ordem direta. Em aplicações mais complexas, ele pode ser estruturado a partir de diferentes componentes, como a definição do papel atribuído ao modelo, a descrição da tarefa, o contexto necessário para executá-la, os dados de entrada, as restrições que devem ser respeitadas, os exemplos de respostas esperadas e o formato de saída (REYNOLDS; MCDONELL, 2021). Em uma tarefa de reparo de vulnerabilidades, por exemplo, o prompt pode informar que o modelo deve atuar como especialista em segurança, apresentar o código analisado, incluir o alerta produzido por uma ferramenta SAST, indicar a CWE relacionada, restringir alterações que não sejam necessárias e exigir que a resposta seja apresentada como um patch ou trecho de código estruturado (KHAN; LIU; GAO, 2025).

A combinação desses elementos reduz a ambiguidade da tarefa e facilita a integração da resposta em um pipeline automatizado. A definição do formato de saída é especialmente importante porque respostas em linguagem natural, embora compreensíveis para uma pessoa, podem dificultar o processamento automático (LIU et al., 2024). Assim, o modelo pode ser orientado a fornecer campos específicos, como a classificação do alerta, a causa da vulnerabilidade, a correção proposta, as linhas modificadas, as justificativas técnicas e eventuais limitações identificadas. A padronização também permite comparar as respostas de diferentes modelos e verificar se todos receberam instruções equivalentes.

Nesse contexto, a engenharia de prompt deve ser compreendida como um processo iterativo e experimental. Pequenas alterações na formulação, na ordem das informações e nos exemplos fornecidos podem modificar significativamente a resposta do modelo (LU et al., 2022).

Convém ainda distinguir engenharia de prompt de *prompt tuning*. A engenharia de prompt utiliza instruções discretas e legíveis escritas em linguagem natural. O termo *prompt tuning*, em seu sentido técnico, costuma representar o aprendizado de vetores contínuos, denominados *soft prompts*, que são ajustados por meio de retropropagação enquanto os parâmetros principais do modelo permanecem congelados (LESTER; AL-RFOU; CONSTANT, 2021). Khan, Liu e Gao (2025) utilizam a expressão *context-aware prompt tuning* para descrever o enriquecimento progressivo de prompts textuais com conhecimento de segurança e de código.

Uma das formas mais simples de organizar a interação com uma LLM é o *zero-shot prompting*. Nesse cenário, o modelo recebe a descrição da tarefa, mas nenhum exemplo de como ela deve ser resolvida. Para o reparo de vulnerabilidades, uma instrução zero-shot poderia apenas solicitar que o modelo identifique uma falha no código e proponha uma versão corrigida (REYNOLDS; MCDONELL, 2021). O resultado dependeria integralmente do conhecimento adquirido durante o treinamento e da capacidade do modelo

de interpretar a instrução fornecida.

No *one-shot prompting*, é incluído um único exemplo resolvido, enquanto no *few-shot prompting* são apresentados alguns exemplos de entrada e saída antes do problema principal. Esses exemplos não modificam os parâmetros internos do modelo. Eles atuam durante a inferência, ajudando a localizar o comportamento desejado entre os padrões já aprendidos e indicando a estrutura esperada da resposta (REYNOLDS; MCDONELL, 2021). Em um cenário de segurança, um exemplo pode apresentar um alerta SAST, um trecho vulnerável e sua correção correspondente, permitindo que o modelo reconheça como os dados do analisador devem ser interpretados.

Entretanto, a utilização de exemplos não garante automaticamente uma resposta melhor. Exemplos pouco representativos podem induzir o modelo a aplicar uma correção inadequada, copiar padrões irrelevantes ou interpretar detalhes acidentais como parte da tarefa (REYNOLDS; MCDONELL, 2021). Além disso, a literatura mostra que a ordem em que exemplos *few-shot* são apresentados pode alterar substancialmente o desempenho do modelo (LU et al., 2022).

Para tarefas que exigem múltiplas etapas de análise, foi proposta a técnica de Cadeia de Pensamento, ou *Chain-of-Thought (CoT)*. Em vez de solicitar diretamente uma resposta final, o prompt orienta o modelo a decompor o problema em passos intermediários. A formulação original do CoT utiliza exemplos que contêm tanto a entrada quanto uma sequência de etapas que conduz à resposta (WEI et al., 2023). Posteriormente, o chamado zero-shot CoT demonstrou que instruções simples de decomposição, como solicitar que o modelo resolva o problema passo a passo, também podem melhorar o desempenho em determinadas tarefas (KOJIMA et al., 2023).

No reparo de vulnerabilidades, a decomposição pode orientar o modelo a executar uma sequência semelhante à utilizada por um especialista: identificar o trecho apontado pelo SAST, determinar se o alerta representa uma vulnerabilidade real, localizar a origem e o destino do fluxo de dados, identificar a causa da falha, selecionar uma estratégia de mitigação, gerar a correção e verificar se a modificação preserva a funcionalidade original (WEI et al., 2023; KHAN; LIU; GAO, 2025). Essa estrutura é mais adequada do que uma solicitação genérica para “corrigir o código”, pois impede que etapas importantes sejam omitidas e facilita a identificação do ponto em que uma resposta incorreta foi produzida.

Contudo, a Cadeia de Pensamento não deve ser interpretada como uma representação necessariamente fiel do processo interno utilizado pelo modelo. A sequência textual pode constituir uma justificativa plausível produzida após a escolha da resposta, sem refletir exatamente os fatores que determinaram a geração. Portanto, o CoT deve ser utilizado como uma estratégia para estruturar a tarefa e produzir etapas verificáveis, e não como uma garantia de transparência ou correção do raciocínio (LESTER; AL-RFOU; CONSTANT, 2021). No contexto deste trabalho, as etapas apresentadas pela LLM deverão ser

confrontadas com os diagnósticos da ferramenta SAST, com testes e com a reanálise do código corrigido.

Uma extensão do CoT é a técnica de *self-consistency*. Em vez de utilizar apenas uma sequência de raciocínio gerada pelo modelo, são produzidas múltiplas respostas para o mesmo problema. Em seguida, seleciona-se a resposta final que apresenta maior concordância entre os diferentes caminhos gerados. Wang et al. (2023) demonstraram que essa estratégia pode melhorar significativamente o desempenho em tarefas de raciocínio, sem exigir treinamento ou ajuste fino adicional.

A aplicação direta de *self-consistency* ao reparo de código, entretanto, exige adaptações. Uma vulnerabilidade pode admitir várias correções textualmente diferentes, mas semanticamente equivalentes. Assim, não seria adequado selecionar uma solução apenas pela igualdade entre os códigos gerados (WANG et al., 2023). Uma possível aplicação consiste em gerar múltiplos candidatos e utilizar critérios externos para compará-los, como compilação, execução de testes, quantidade de alertas SAST restantes, ausência de novas vulnerabilidades e preservação do comportamento esperado. Nesse cenário, a ferramenta de análise e os testes assumem o papel de mecanismos de seleção e validação das alternativas.

Outra estratégia relevante para fornecer informações ao modelo é a Geração Aumentada por Recuperação, ou *Retrieval-Augmented Generation* (RAG). Diferentemente de uma técnica composta apenas pela escrita de instruções, o RAG constitui uma arquitetura que conecta a LLM a fontes externas de conhecimento. Sua proposta original combina a memória paramétrica, correspondente às informações representadas nos parâmetros do modelo, com uma memória não paramétrica, composta por documentos armazenados externamente e recuperados no momento da consulta (KLESEL; WITTMANN, 2025).

Um pipeline de RAG pode ser dividido em uma etapa de preparação e outra de consulta. Na preparação, os documentos são coletados, processados e separados em fragmentos, frequentemente denominados *chunks*. Cada fragmento é convertido em uma representação vetorial por um modelo de *embeddings* e armazenado em uma base vetorial. Durante a consulta, a entrada do usuário também é transformada em um vetor, permitindo que o recuperador calcule a similaridade entre a pergunta e os documentos armazenados. Os fragmentos mais relevantes são selecionados, incorporados ao prompt e enviados à LLM junto com a tarefa original. Por fim, o modelo gera uma resposta condicionada tanto ao seu conhecimento interno quanto ao conteúdo recuperado (KLESEL; WITTMANN, 2025).

As três etapas centrais desse processo são a recuperação, o enriquecimento e a geração. Na recuperação, são localizados os documentos considerados mais relevantes para a consulta. No enriquecimento, os documentos selecionados são organizados e anexados à instrução. Na geração, a LLM produz a resposta utilizando o contexto ampliado (KLESEL; WITTMANN, 2025). No domínio deste trabalho, a consulta pode ser formada a partir da

linguagem de programação, da regra emitida pelo SAST, da categoria CWE, das funções utilizadas e do tipo de vulnerabilidade detectada.

A base de conhecimento pode incluir descrições do CWE, recomendações do OWASP, regras e documentações do SonarQube, guias de programação segura, documentação oficial da linguagem e do framework analisado, além de exemplos de reparos previamente validados. Dessa forma, quando uma ferramenta SAST identificar uma falha criptográfica, por exemplo, o sistema poderá recuperar informações sobre algoritmos recomendados, APIs atuais e formas adequadas de armazenamento de chaves. Para uma vulnerabilidade de injeção, poderão ser recuperadas recomendações relacionadas a consultas parametrizadas, validação de entrada e neutralização de dados.

Embora o RAG possa melhorar a fundamentação das respostas e fornecer informações que não estavam presentes no treinamento do modelo, ele não elimina automaticamente as alucinações. O resultado depende da qualidade da base documental, da divisão dos documentos, do modelo de *embeddings*, da formulação da consulta e da capacidade do recuperador de selecionar informações realmente relevantes. Caso um documento incorreto, desatualizado ou pouco relacionado seja recuperado, ele poderá induzir a LLM a gerar uma correção inadequada (KLESEL; WITTMANN, 2025). Portanto, os documentos utilizados devem ser provenientes de fontes confiáveis e sua origem deve ser mantida para permitir rastreabilidade e verificação.

No domínio de segurança, a importância do contexto fornecido pelo prompt é demonstrada por (KHAN; LIU; GAO, 2025). Ao avaliarem o GitHub Copilot em 156 trechos com vulnerabilidades relacionadas a estouro de buffer, os autores observaram uma taxa de 76% de sucesso na detecção, mas apenas 15% no reparo quando nenhum conhecimento adicional era fornecido. Informar somente que o código possuía uma vulnerabilidade elevou a taxa de reparo para 20%, enquanto a inclusão do tipo de CWE aumentou o resultado para 31%. Quando o prompt passou a incorporar progressivamente informações sobre segurança e sobre o contexto do código, a taxa de sucesso alcançou 63%.

Neste trabalho, a engenharia de prompt será utilizada para organizar essas diferentes fontes de informação. O prompt poderá ser composto pela descrição da tarefa, pelo código vulnerável, pelos dados do alerta SAST, pela regra violada, pela categoria CWE, pelo contexto recuperado por meio do RAG e pelas restrições de preservação da funcionalidade. Exemplos *few-shot* poderão orientar o formato da resposta e demonstrar correções previamente validadas, enquanto uma estratégia de decomposição conduzirá a LLM pelas etapas de triagem, análise da causa, geração do reparo e revisão da modificação.

A resposta do modelo deverá ser estruturada para permitir sua utilização automática pelo pipeline. Além do código corrigido, poderão ser solicitados a decisão sobre a validade do alerta, o tipo de modificação realizada e as suposições necessárias para a solução. Entretanto, nenhuma justificativa produzida pela LLM será considerada sufici-

ente para validar o reparo. O código será novamente submetido à ferramenta SAST e aos mecanismos de verificação funcional definidos no experimento.

Assim, o enriquecimento do prompt, o RAG e a decomposição orientada não substituem a validação externa. Essas técnicas buscam aumentar a quantidade e a qualidade das informações disponíveis durante a geração. A confiabilidade final será obtida pela combinação entre contexto técnico, geração orientada e feedback de ferramentas externas. Nesse fluxo, o alerta do SAST deixa de ser apenas uma mensagem textual e passa a integrar um processo iterativo no qual o diagnóstico, o conhecimento recuperado, a correção gerada e os resultados da reanálise são utilizados conjuntamente para produzir e validar o reparo.

2.3 Vulnerabilidades de Software e OWASP 10

Uma vulnerabilidade de software pode ser compreendida como uma fraqueza presente na arquitetura, no projeto, na implementação ou na configuração de um sistema que pode ser explorada por um agente malicioso para violar alguma propriedade de segurança. A exploração dessas falhas pode comprometer a confidencialidade, a integridade ou a disponibilidade das informações e dos serviços oferecidos pelo sistema. Dependendo da vulnerabilidade, as consequências podem incluir acesso indevido a dados, execução não autorizada de código, alteração de informações e interrupção de serviços (SáNCHEZ et al., 2020).

Entre as consequências mais relevantes encontram-se a negação de serviço, do inglês *Denial of Service* (DoS), e a escalação de privilégios. Em um ataque de negação de serviço, uma falha é explorada para tornar determinado recurso ou serviço indisponível aos usuários legítimos, por exemplo, provocando o encerramento da aplicação ou consumindo excessivamente seus recursos. Já na escalação de privilégios, o atacante obtém permissões superiores às que deveria possuir, podendo passar de um usuário comum para um usuário administrativo e, assim, ler, modificar ou excluir informações protegidas. Sánchez et al. (2020) identificaram essas consequências entre as mais relevantes nas vulnerabilidades de maior impacto analisadas em seu estudo.

Para organizar a identificação e a comunicação dessas falhas, a área de segurança utiliza um ecossistema composto por diferentes bases e sistemas de pontuação. Entre os principais componentes desse ecossistema encontram-se o *Common Vulnerabilities and Exposures* (CVE), a *National Vulnerability Database* (NVD), o *Common Weakness Enumeration* (CWE) e o *Common Vulnerability Scoring System* (CVSS). Embora esses mecanismos sejam relacionados, cada um desempenha uma função diferente no gerenciamento de vulnerabilidades.

O ecossistema de gerenciamento de vulnerabilidades é composto por mecanismos

complementares. O Common Vulnerabilities and Exposures (CVE) identifica vulnerabilidades publicamente divulgadas por meio de códigos únicos, permitindo que pesquisadores, fabricantes e ferramentas se refiram à mesma falha (CVE Program, 2026). A National Vulnerability Database (NVD), mantida pelo NIST, reúne esses registros e os enriquece com informações como produtos afetados, referências, configurações vulneráveis e métricas de severidade (National Institute of Standards and Technology, 2026). Já o Common Weakness Enumeration (CWE) classifica os tipos de fraqueza que podem originar vulnerabilidades, permitindo que diferentes CVEs sejam associados a uma mesma causa estrutural (MITRE Corporation, 2024).

O Common Vulnerability Scoring System (CVSS) complementa esse ecossistema ao atribuir uma pontuação de severidade entre 0 e 10, classificada como nenhuma, baixa, média, alta ou crítica. Assim, enquanto o CVE identifica uma vulnerabilidade concreta, o CWE descreve o tipo de fraqueza, a NVD agrega informações sobre o registro e o CVSS comunica sua severidade técnica. Entretanto, essa pontuação não representa, isoladamente, o risco para uma organização, que também depende da exposição do componente, da importância do ativo, da possibilidade de exploração, dos controles existentes e do impacto para o negócio (Forum of Incident Response and Security Teams, 2023).

No contexto específico de aplicações web, uma das principais referências para conscientização e orientação é o OWASP Top 10. Mantido pela Open Worldwide Application Security Project (OWASP), o documento apresenta categorias que representam um consenso amplo sobre os riscos mais críticos enfrentados por aplicações web. O OWASP Top 10 não deve ser interpretado como uma lista completa de todas as vulnerabilidades existentes nem como uma norma de certificação. Seu objetivo principal é ampliar a conscientização, apoiar treinamentos e orientar organizações na identificação e prevenção de classes recorrentes de problemas de segurança (OWASP Foundation, 2025g).

A construção da edição de 2025 foi fundamentada em uma combinação de dados empíricos e avaliação da comunidade especializada. Organizações de segurança forneceram dados provenientes de testes realizados em mais de 2,8 milhões de aplicações. Diferentemente da edição de 2021, a coleta de 2025 não restringiu previamente quais CWEs poderiam ser enviadas, resultando na análise de 589 tipos de fraquezas. Para cada CWE, o OWASP considerou a quantidade de aplicações testadas e a quantidade de aplicações em que pelo menos uma ocorrência da fraqueza foi identificada (OWASP Foundation, 2025g).

Essa metodologia considera principalmente a prevalência da fraqueza entre as aplicações, e não o número absoluto de ocorrências dentro de uma mesma aplicação. Dessa forma, um sistema contendo quatro instâncias de determinada CWE e outro contendo quatro mil instâncias são contabilizados igualmente para a medição de prevalência. Essa decisão reduz distorções entre testes manuais, que frequentemente registram uma falha

apenas uma vez, e ferramentas automatizadas, que podem registrar separadamente cada ocorrência encontrada (OWASP Foundation, 2025g).

Além dos dados quantitativos, o OWASP utiliza uma pesquisa com profissionais de desenvolvimento e segurança. A edição de 2025 é descrita como informada por dados, mas não cegamente dirigida por eles. Isso ocorre porque os resultados fornecidos pelas ferramentas representam, em grande parte, vulnerabilidades que a indústria já sabe detectar em escala. Novos riscos podem demorar anos até serem incorporados aos mecanismos automatizados de teste (OWASP Foundation, 2025g).

Outro aspecto relevante da edição de 2025 é a priorização das causas raízes das vulnerabilidades em vez de seus sintomas. Um sintoma representa o efeito observável de uma falha, enquanto a causa raiz corresponde à condição técnica que permitiu sua ocorrência (OWASP Foundation, 2025g). Essa escolha possui importância prática para o reparo automático. Uma descrição baseada somente no sintoma pode indicar que dados foram expostos, mas fornece poucas informações sobre qual modificação deve ser realizada no código. Já a identificação da causa raiz pode indicar a necessidade de substituir um algoritmo obsoleto, remover uma chave inserida diretamente no código ou utilizar uma função segura para armazenamento de senhas, por exemplo. Dessa maneira, categorias orientadas à causa oferecem informações mais adequadas para ferramentas SAST e modelos de linguagem responsáveis por sugerir correções (KHAN; LIU; GAO, 2025).

Entre as categorias que compõem o OWASP Top 10 de 2025, a Quebra de Controle de Acesso, as Falhas Criptográficas, as vulnerabilidades de Injeção, as Falhas de Autenticação e o Tratamento Incorreto de Condições Excepcionais merecem atenção por afetarem mecanismos essenciais de segurança das aplicações e por poderem comprometer propriedades como confidencialidade, integridade, disponibilidade e autenticidade. Essas categorias representam diferentes formas de fragilidade no desenvolvimento de software e são apresentadas a seguir com maior detalhamento quanto às suas características, formas de ocorrência e possíveis impactos (OWASP Foundation, 2025b; OWASP Foundation, 2025c; OWASP Foundation, 2025d; OWASP Foundation, 2025e; OWASP Foundation, 2025f).

A A01:2025 (Quebra de Controle de Acesso) ocorre quando as restrições sobre as ações permitidas aos usuários não são aplicadas corretamente, possibilitando o acesso a recursos de terceiros, páginas administrativas, manipulação de tokens e elevação indevida de privilégios; essa categoria também passou a incluir ataques de *Server-Side Request Forgery* (SSRF).

A A04:2025 (Falhas Criptográficas) abrange problemas como ausência de criptografia, uso de algoritmos ou protocolos obsoletos, exposição de chaves, armazenamento inadequado de senhas e geração de valores previsíveis, podendo resultar na exposição de dados sensíveis e no comprometimento do sistema.

A A05:2025 (Injeção) ocorre quando entradas não confiáveis são enviadas a interpretadores sem validação, neutralização ou parametrização adequadas, incluindo ataques como *SQL Injection, NoSQL Injection*, injeção de comandos e *Cross-site Scripting* (XSS); sua prevenção envolve consultas parametrizadas, validação de entradas e neutralização contextual dos dados.

A A07:2025 (Falhas de Autenticação) envolve o reconhecimento indevido de usuários, a proteção inadequada de credenciais e sessões, o uso de senhas incorporadas ao código, credenciais padrão e mecanismos vulneráveis à fixação de sessão, sendo recomendada a utilização de *frameworks* consolidados e autenticação multifator.

Por fim, a A10:2025 (Tratamento Incorreto de Condições Excepcionais) está relacionada a erros não tratados, exceções não capturadas, falhas lógicas e situações em que o sistema falha de maneira insegura, podendo provocar vazamento de informações, corrupção do estado da aplicação, transações inconsistentes ou negação de serviço (OWASP Foundation, 2025g).

Essa diversidade de vulnerabilidades possibilita analisar não apenas se a LLM remove o alerta produzido pela ferramenta SAST, mas também se ela compreende a causa da vulnerabilidade e gera uma correção que mantém a funcionalidade originalmente esperada.

2.4 Ferramentas de Análise Estática de Segurança (SAST)

O Teste Estático de Segurança de Aplicações, do inglês *Static Application Security Testing* (SAST), compreende um conjunto de técnicas e ferramentas utilizadas para examinar o código-fonte de um sistema sem executar a aplicação. Seu objetivo é identificar e relatar fraquezas de implementação que possam originar vulnerabilidades, como manipulação insegura de entradas, uso inadequado de memória, chamadas perigosas de funções, configurações criptográficas frágeis e violações de práticas de programação segura. Por ter acesso direto ao código, essa abordagem também é classificada como uma forma de teste de caixa-branca (OWASP Foundation, 2026c; National Institute of Standards and Technology, 2021).

A análise estática é normalmente aplicada durante a fase de implementação do ciclo de desenvolvimento seguro de software. Como não exige que o sistema esteja implantado e em execução, ela pode ser executada desde as primeiras versões do código e repetida a cada alteração realizada pelos desenvolvedores. Essa característica favorece a adoção do princípio de *shift-left*, no qual atividades de segurança são antecipadas para etapas iniciais do desenvolvimento, reduzindo o custo de correções que, de outra forma, seriam descobertas apenas durante testes finais ou após a implantação do sistema (OWASP Foundation, 2026c).

As ferramentas SAST também podem ser integradas a ambientes de desenvolvimento, repositórios de código e pipelines de integração contínua. Dessa forma, um novo trecho pode ser analisado durante sua escrita, em uma solicitação de integração ou antes da publicação de uma nova versão. A repetibilidade representa uma das principais vantagens da análise estática, pois permite examinar grandes quantidades de código de forma automática e executar verificações frequentes, como em cada *commit* e *merge request*. Entretanto, essa automação não significa que todos os alertas correspondam necessariamente a vulnerabilidades reais (IRANMANESH et al., 2025).

O funcionamento de uma ferramenta SAST pode envolver diferentes técnicas derivadas de compiladores e de análise de programas. Essas técnicas são utilizadas em conjunto para representar a estrutura do código, acompanhar as possíveis formas de execução e verificar se determinados padrões violam regras de segurança. Entre as técnicas mais relevantes estão a análise léxica, a análise de fluxo de controle, a análise de fluxo de dados e a análise de propagação de dados não confiáveis, conhecida como *taint analysis* (OWASP Foundation, 2026c).

A análise léxica transforma o código-fonte em unidades menores chamadas tokens. Palavras-chave, identificadores, operadores, literais e símbolos são separados e classificados, permitindo que a ferramenta deixe de tratar o programa como uma sequência comum de caracteres e passe a reconhecer sua estrutura sintática. Essa etapa permite detectar padrões locais, como a utilização de funções consideradas perigosas, credenciais inseridas diretamente no código, algoritmos criptográficos obsoletos ou construções incompatíveis com determinadas regras de programação segura (OWASP Foundation, 2026c).

Entretanto, vulnerabilidades complexas não podem ser detectadas apenas pela procura de palavras ou padrões isolados. Para compreender os diferentes caminhos que um programa pode seguir, as ferramentas podem construir uma representação de seu fluxo de controle. Nessa representação, o código é dividido em blocos básicos, formados por sequências de instruções executadas consecutivamente, e as transições possíveis entre esses blocos são organizadas em um Grafo de Fluxo de Controle, do inglês Control Flow Graph (CFG). Esse grafo permite representar condições, repetições, chamadas e desvios que influenciam a execução do programa (OWASP Foundation, 2026c).

A análise de fluxo de dados permite que ferramentas SAST acompanhem como valores são definidos, modificados e utilizados ao longo dos possíveis caminhos do programa. Uma aplicação importante dessa técnica é a *taint analysis*, que identifica dados provenientes de fontes não confiáveis e rastreia sua propagação até operações sensíveis, denominadas *sinks*. Caso esses dados alcancem um destino, como uma consulta SQL, uma execução de comando ou a geração de uma página web, sem passar por validação, sanitização ou neutralização adequada, a ferramenta pode emitir um alerta de vulnerabilidade. Dessa forma, a análise considera a relação entre fonte, sanitizador e destino, permitindo

identificar não apenas a linha problemática, mas todo o caminho percorrido pela entrada. Essas técnicas possibilitam detectar antecipadamente vulnerabilidades como injeção SQL, *Cross-site Scripting* e uso de funções inseguras, sem exigir a execução da aplicação ou a preparação de entradas específicas de teste (OWASP Foundation, 2026c).

Apesar dessas vantagens, a análise estática apresenta limitações. Um resultado produzido pela ferramenta representa a identificação de uma condição potencialmente problemática, e não necessariamente a comprovação de que ela poderá ser explorada em uma aplicação real. Um falso positivo ocorre quando a ferramenta aponta uma possível vulnerabilidade que, após análise, não representa um risco real. Isso pode acontecer, por exemplo, quando a ferramenta não reconhece um mecanismo de validação personalizado utilizado pela aplicação ou não consegue acompanhar o comportamento de um componente externo (IRANMANESH et al., 2025).

O ecossistema de análise estática inclui ferramentas comerciais, acadêmicas e de código aberto, com diferentes níveis de especialização. Algumas possuem cobertura para diversas linguagens e tipos de fraqueza, enquanto outras são direcionadas a uma única linguagem, plataforma ou categoria de vulnerabilidade (National Institute of Standards and Technology, 2021). A escolha de uma ferramenta deve considerar fatores como as linguagens suportadas, os tipos de vulnerabilidade detectados, a necessidade de o projeto estar completamente compilável, a integração com IDEs e pipelines, os formatos de relatório disponíveis e a possibilidade de personalizar regras.

Em ferramentas de análise estática, uma regra representa uma condição ou prática que deve ser verificada no código. Quando o analisador encontra uma violação, é criado um *issue* associado à regra responsável pela detecção. As regras podem ser organizadas de acordo com aspectos de segurança, confiabilidade e manutenibilidade (SonarSource, 2026).

As regras possuem ainda estados que indicam seu grau de maturidade. Uma regra classificada como *Ready* é considerada adequada para utilização em produção; uma regra Beta ainda não recebeu feedback suficiente e pode apresentar falsos positivos ou negativos; e uma regra *Deprecated* não deve mais ser utilizada quando existe uma alternativa mais atual ou precisa. As regras também podem ser associadas a níveis de severidade, definidos a partir do impacto potencial e da probabilidade de ocorrência ou exploração do problema (SonarSource, 2026).

No pipeline proposto, uma primeira execução da SAST será utilizada para estabelecer o estado inicial do código. Para cada issue relacionado à segurança, poderão ser coletados dados como o identificador e a descrição da regra, o arquivo e a linha afetados, a mensagem apresentada, a severidade, as classificações CWE e OWASP, as localizações secundárias e os fluxos entre fontes e destinos. Esses dados formarão o contexto técnico fornecido à LLM.

Após a geração do reparo, o projeto será analisado novamente pela SAST. Essa nova execução permitirá verificar se o issue original deixou de ser identificado e se a alteração introduziu novos alertas. Entretanto, o desaparecimento do alerta não será considerado, isoladamente, prova suficiente de que o reparo foi bem-sucedido. A ferramenta pode deixar de detectar uma vulnerabilidade mesmo que ela permaneça no código, e a LLM pode eliminar o alerta comprometendo a funcionalidade da aplicação. Dessa forma, a validação deverá combinar a reanálise estática com compilação, testes funcionais e comparação do comportamento esperado.

Nota-se, portanto, que a principal contribuição do SAST para o pipeline não se limita a informar que existe um possível problema. A ferramenta fornece um diagnóstico estruturado que pode reduzir a ambiguidade da tarefa de reparo. Informações sobre a regra violada, o tipo de fraqueza, a localização e o fluxo de dados ajudam a LLM a identificar a causa da vulnerabilidade e a escolher uma correção compatível com o código analisado.

2.5 Trabalhos Relacionados

A literatura recente tem demonstrado que o uso isolado de Modelos de Linguagem de Grande Escala (abordagem *single-pass*) é insuficiente para a correção confiável de vulnerabilidades de software. Estudos apontam que, sem o fornecimento de um contexto adequado, os modelos falham em compreender a causa raiz do problema de segurança e frequentemente comprometem a funcionalidade do sistema ao tentar corrigi-lo (DAI; XU; TAO, 2025).

Para otimizar o controle e a eficácia dessas ferramentas, Reynolds e McDonell (2021) introduziram o conceito formal de programação de prompts demonstrando que a forma como a instrução é estruturada em linguagem natural impacta drasticamente o desempenho do modelo. Os autores ressaltam a importância de técnicas nas quais a inteligência artificial é induzida a decompor um problema complexo em etapas menores antes de um veredito, evitando respostas precipitadas e permitindo um processamento lógico mais eficiente.

Diante deste contexto de reparos, novas metodologias passaram a integrar técnicas de Geração Aumentada por Recuperação (RAG) e Cadeia de Pensamento (CoT). O estudo de Tao et al. (2025) combinou essas estratégias para orientar o modelo a primeiro compreender a semântica do código para, em seguida, analisar os trechos vulneráveis e gerar sugestões de reparo. A pesquisa demonstrou que integrar a recuperação de conhecimentos externos (RAG) juntamente ao raciocínio em etapas (CoT) alcança um desempenho superior em diversas métricas na geração de instruções e soluções de segurança.

Paralelamente à evolução da engenharia de prompts, os estudos evidenciam que os modelos generativos atingem sua máxima eficácia quando inseridos em pipelines iterativos

com ferramentas de análise externa. Sriram et al. (2026) desenvolveram um fluxo de auto-reparo no qual um LLM refina continuamente suas próprias respostas utilizando o feedback diagnóstico de ferramentas SAST (como o CodeQL). Esse estudo comprovou que a integração iterativa de múltiplas ferramentas conseguiu reduzir a ocorrência de vulnerabilidades críticas geradas em até 96% .

Diante das lacunas e dos resultados apresentados por ests trabalhos, esta pesquisa fundamenta-se na integração dessas tecnologias. Ao combinar o diagnóstico de uma ferramenta SAST com modelos de linguagem comerciais utilizando técnicas de RAG e Cadeia de Pensamento (CoT), o estudo propõe um fluxo iterativo capaz de interpretar os alertas gerados e sugerir correções mais precisas para vulnerabilidades associadas ao OWASP Top 10. Com isso, busca-se alcançar um método que contribua para a correção das falhas sem comprometer negativamente o código.

3 Metodologia

Este capítulo apresenta os procedimentos metodológicos propostos para avaliar a eficácia de um *pipeline* automatizado de correção de vulnerabilidades em código-fonte com o apoio de Modelos de Linguagem de Grande Escala, ferramentas de Análise Estática de Segurança e estratégias de contextualização.

A pesquisa será conduzida sobre o OWASP Juice Shop, uma aplicação web *open source* deliberadamente vulnerável e utilizada como referência em treinamentos, demonstrações de segurança e avaliação de ferramentas de segurança. Serão consideradas as vulnerabilidades de segurança efetivamente identificadas por uma ferramenta SAST no projeto selecionado. Essa decisão aproxima o experimento de um cenário prático de desenvolvimento, no qual possui regras de negócio, a arquitetura do sistema precisa ser compreendida e as correções devem respeitar as dependências, os testes e o funcionamento existente da aplicação.

A escolha por este projeto permite avaliar o *pipeline* em uma aplicação de grande porte, composta por múltiplos módulos, dependências e suíte de testes, proporcionando um cenário mais representativo do que exemplos sintéticos ou casos isolados.

3.1 Plano Metodológico

Esta pesquisa é um estudo empírico de natureza aplicada, com abordagem predominantemente quantitativa. Em relação aos objetivos, tem caráter exploratório e avaliativo, pois busca investigar a eficácia de diferentes estratégias de *prompt* na correção automatizada de vulnerabilidades de segurança. Em relação aos procedimentos, a pesquisa será conduzida como um estudo experimental sobre uma aplicação de referência, na qual um *pipeline* automatizado será executado repetidas vezes para avaliar o desempenho das estratégias propostas, com base em métricas definidas previamente.

O estudo será conduzido a partir da implementação e avaliação de um *pipeline* automatizado de correção de vulnerabilidades. Esse *pipeline* receberá como entrada um projeto com vulnerabilidades identificadas por uma ferramenta SAST, coletará informações sobre as *issues* encontradas, enriquecerá o contexto fornecido ao modelo de linguagem, solicitará uma correção à LLM, aplicará a modificação gerada e, em seguida, executará novamente a análise estática e os testes automatizados.

O fluxo metodológico geral será composto pelas seguintes etapas:

1. Preparação do ambiente de execução do projeto;

2. *Pipeline*

3. Consolidação dos resultados em um conjunto estruturado de dados;

4. Análise comparativa dos resultados obtidos.

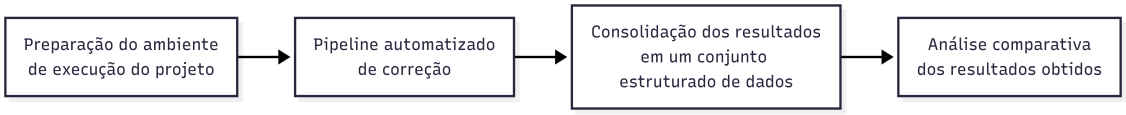


Figura 1 – Visão geral do fluxo metodológico adotado na pesquisa.

Legenda: A etapa *Pipeline automatizado de correção* está detalhada na Figura 2.

A metodologia parte da premissa de que uma correção de vulnerabilidade não deve ser considerada bem-sucedida apenas porque a ferramenta SAST deixou de apontar uma issue. Uma alteração pode remover o alerta sem corrigir a causa real da falha, ou pode eliminar a vulnerabilidade comprometendo a funcionalidade esperada do sistema. Por esse motivo, a avaliação combinará três critérios principais: o desaparecimento da issue na reanálise estática, a aprovação dos testes automatizados após a aplicação da correção e a não introdução de novas issues de segurança.

Assim, uma correção será considerada bem-sucedida apenas quando atender simultaneamente aos critérios de segurança, funcionalidade e não regressão de segurança.

3.2 Contexto de aplicação do experimento

O experimento será conduzido sobre o OWASP Juice Shop, uma aplicação web *open source* deliberadamente vulnerável e mantida no ecossistema da OWASP. A escolha do OWASP Juice Shop como objeto de estudo foi motivada por quatro fatores principais: sua disponibilidade como projeto *open source*, sua adoção pela comunidade de segurança, sua implementação em tecnologias amplamente utilizadas no desenvolvimento web moderno e sua concentração de vulnerabilidades documentadas e associadas ao OWASP Top 10 (OWASP Foundation, 2026a).

A prova de conceito inicial utilizou códigos vulneráveis pequenos e isolados para validar o funcionamento inicial do *pipeline*. No TCC 2, essa abordagem será ampliada para um projeto mais completo, contendo múltiplos arquivos, estrutura modular, testes automatizados e dependências externas. Essa ampliação permitirá avaliar se o *pipeline* continua funcional em um cenário mais próximo de um projeto real.

Para a correção do código vulnerável será utilizado o modelo `gpt-4.1-mini`. A escolha do modelo `gpt-4.1-mini` foi definida a partir de critérios de viabilidade expe-

rimental, custo de execução e adequação ao fluxo automatizado proposto. Como o *pipeline* realiza múltiplas chamadas à API para diferentes vulnerabilidades e estratégias de *prompt*, o consumo de tokens representa um fator relevante para a condução do estudo. Nesse sentido, a utilização de um modelo menor da família GPT-4.1 permite reduzir o custo das execuções, mantendo recursos importantes para o experimento, como ampla janela de contexto, suporte a saídas estruturadas e capacidade de seguir instruções em tarefas envolvendo código (OPENAI, 2026). Além disso, a implementação do *pipeline* será parametrizada, permitindo substituir o modelo por versões anteriores ou posteriores em execuções futuras, o que facilita testes comparativos sem alterar a lógica central do experimento.

Além disso, a escolha por um modelo da OpenAI se justifica pela representatividade da família GPT no cenário atual de uso de LLMs. Como o objetivo do trabalho é avaliar um pipeline de reparo automático apoiado por um modelo comercial de uso amplo, optou-se por utilizar uma tecnologia já consolidada entre usuários, desenvolvedores e organizações, em vez de um modelo experimental ou de adoção restrita. Essa escolha aumenta a relevância prática do estudo, pois permite analisar o comportamento de um modelo pertencente à família de LLMs mais difundida no uso cotidiano e profissional. A popularidade do ChatGPT é evidenciada por dados de mercado que o colocam como o site mais acessado na categoria de ferramentas e chatbots de IA, além de relatórios que indicam grande volume de uso semanal e forte presença em ambientes profissionais. No contexto específico de desenvolvimento de software, a pesquisa *Stack Overflow Developer Survey 2025* também aponta o ChatGPT como uma das ferramentas de IA mais utilizadas por desenvolvedores, sendo citado por 81,7% dos respondentes que utilizam agentes e assistentes de IA. (Similarweb, 2026; Stack Overflow, 2025).

3.3 Objeto de estudo: OWASP Juice Shop

O OWASP Juice Shop é implementado principalmente em JavaScript e TypeScript, utilizando Node.js, Express e Angular. Por esse motivo, a metodologia considerará ferramentas, regras SAST, testes e estratégias de correção compatíveis com essas linguagens.

Na versão utilizada nesta pesquisa (v20.1.1), o OWASP Juice Shop reúne 113 desafios distribuídos em 16 categorias de vulnerabilidades, abrangendo problemas como exposição de dados sensíveis, injeção, quebra de controle de acesso, validação imprópria de entrada e *Cross-Site Scripting* (XSS). Essas categorias são derivadas de padrões amplamente reconhecidos pela comunidade, incluindo o OWASP Top 10, o OWASP ASVS, o OWASP API Security Top 10 e a Common Weakness Enumeration (CWE), tornando a aplicação uma base adequada para pesquisas em segurança de software.

Além dos desafios de exploração, o Juice Shop disponibiliza 33 Coding Challenges, nos quais o usuário localiza o trecho vulnerável do código e identifica a alternativa de correção mais adequada. Embora, nesta pesquisa, as correções sejam geradas automaticamente por um pipeline baseado em LLM, a existência desses desafios evidencia que a aplicação foi projetada para apoiar estudos sobre identificação e remediação de vulnerabilidades, oferecendo um ambiente reprodutível e amplamente documentado para avaliação da abordagem proposta.

3.4 Seleção das vulnerabilidades de segurança

A seleção das vulnerabilidades ocorrerá a partir dos alertas produzidos pela ferramenta SAST durante a análise inicial do OWASP Juice Shop. Como a aplicação é deliberadamente vulnerável, espera-se que parte dos alertas esteja relacionada a falhas conhecidas ou compatíveis com categorias do OWASP Top 10. Dessa forma, o conjunto de vulnerabilidades analisado será composto pelos alertas efetivamente identificados pela ferramenta SAST durante a execução inicial do *pipeline*.

Após a execução inicial da ferramenta SAST, as vulnerabilidades detectadas serão submetidas ao *pipeline* de correção. Para cada *issue* identificada, serão aplicadas as diferentes estratégias de *prompt* propostas neste trabalho, permitindo avaliar o desempenho da LLM na remediação de diferentes tipos de vulnerabilidades presentes na aplicação.

Embora o OWASP Juice Shop contemple vulnerabilidades associadas ao OWASP Top 10, o experimento não partirá da seleção manual de categorias específicas. As tentativas de correção serão realizadas sobre todos os alertas produzidos pela ferramenta SAST. Sempre que a ferramenta fornecer mapeamentos para CWE, OWASP ou outras regras de segurança, essas informações serão registradas e utilizadas posteriormente na análise dos resultados.

As vulnerabilidades identificadas serão documentadas em um arquivo de metadados contendo:

1. identificador da *issue*;
2. ferramenta SAST responsável pela detecção;
3. regra acionada;
4. severidade;
5. arquivo e linha afetados;
6. mensagem da ferramenta;

7. CWE ou categoria OWASP associada, quando disponível;
8. trecho de código relacionado;
9. estratégia de validação utilizada após a correção.

3.5 Ferramenta de análise estática

A análise estática será utilizada para identificar vulnerabilidades antes e depois da aplicação das correções. A ferramenta SAST será responsável por produzir os diagnósticos iniciais que servirão como entrada para o *pipeline*, além de permitir a verificação posterior do desaparecimento das *issues*.

Na prova de conceito, verificou-se a necessidade de avaliar a adequação da ferramenta SAST ao contexto Python e aos tipos de vulnerabilidade analisados. Por isso, no TCC 2, a escolha da ferramenta considerará a capacidade real de detectar as vulnerabilidades. Será utilizada a ferramenta Semgrep como ferramenta SAST principal, por oferecer suporte a JavaScript e TypeScript, além de permitir a exportação dos resultados em formato JSON para integração com o *pipeline*.

Para cada execução da análise estática, serão coletados os seguintes dados:

1. Identificador da issue;
2. Ferramenta utilizada;
3. Regra acionada;
4. Severidade;
5. Arquivo e linha afetados;
6. Mensagem da ferramenta;
7. Categoria CWE ou OWASP, quando disponível;
8. Trecho de código relacionado;
9. Data e identificador da execução.

A análise estática inicial será executada antes da chamada à LLM. Em seguida, após a aplicação da correção gerada, a ferramenta será executada novamente para verificar se a *issue* original deixou de ser detectada e se novas *issues* foram introduzidas.

3.6 Estratégias de prompt

O experimento avaliará diferentes estratégias de *prompt* para verificar como a formulação da instrução influencia a qualidade das correções geradas. A comparação entre *prompts* é essencial para observar se estratégias mais estruturadas, com contexto adicional e raciocínio guiado, produzem correções melhores do que instruções simples. Serão avaliadas, inicialmente, quatro estratégias.

A primeira será o ***prompt* simples**, no qual o modelo recebe o trecho de código do OWASP Juice Shop associado ao alerta SAST e uma instrução direta para identificar e corrigir a vulnerabilidade.

A segunda será o ***prompt* com raciocínio guiado**, inspirado na ideia de *Chain-of-Thought* discutida no referencial teórico. Neste trabalho, essa estratégia não será utilizada como forma de acessar ou validar o raciocínio interno do modelo, mas como uma maneira de estruturar a tarefa em etapas explícitas. Assim, o modelo será orientado a identificar a vulnerabilidade, explicar sua causa, descrever o impacto, propor uma correção, justificar a alteração e indicar possíveis cuidados adicionais.

A terceira será o ***prompt* com RAG simplificado**, no qual o modelo recebe, além do código e do alerta SAST, um documento de contexto recuperado automaticamente a partir da categoria da vulnerabilidade identificada.

A quarta será o ***prompt* com RAG simplificado e raciocínio guiado**, combinando o contexto técnico recuperado com uma estrutura de resposta em etapas.

Para permitir integração automática com o pipeline, as respostas da LLM serão solicitadas em formato estruturado JSON. O formato esperado deverá conter campos:

1. Vulnerabilidade identificada;
2. Causa da vulnerabilidade;
3. Estratégia de correção;
4. Código corrigido ou *patch*;
5. Justificativa técnica;
6. Observações sobre possíveis limitações da correção.

A padronização da saída é necessária para que o *pipeline* consiga extrair automaticamente o código corrigido, aplicar a alteração e registrar os dados da execução.

3.7 Estratégia de RAG simplificado

Neste trabalho, o RAG será implementado de forma simplificada, por meio da recuperação automática de documentos de contexto associados à categoria da vulnerabilidade identificada pela ferramenta SAST. Diferentemente de arquiteturas RAG completas baseadas em *embeddings*, bancos vetoriais e mecanismos de ranqueamento semântico, a abordagem proposta utilizará uma base documental controlada e indexada por categorias de vulnerabilidade.

Essa decisão metodológica foi adotada por questões de viabilidade e alinhamento com o escopo do trabalho. O objetivo da pesquisa não é desenvolver uma arquitetura de RAG sofisticada, mas avaliar se a inclusão de contexto técnico adicional no *prompt* contribui para melhorar a qualidade das correções geradas pela LLM. Assim, a recuperação será baseada em um mapeamento direto entre o tipo de vulnerabilidade identificado e o documento de apoio correspondente.

A base de conhecimento será composta por arquivos em formato *Markdown*, organizados em uma pasta específica do projeto. Cada arquivo conterá informações técnicas sobre uma categoria de vulnerabilidade, incluindo descrição da falha, classificação OWASP, CWE relacionada, formas comuns de exploração, recomendações de mitigação e exemplos de código vulnerável e corrigido.

A estrutura inicial da base poderá seguir o seguinte formato, podendo ser ajustada:

```
contextos_rag/
  injection.md
  broken_access_control.md
  cryptographic_failures.md
  authentication_failures.md
  security_misconfiguration.md
  cross_site_scripting.md
  vulnerable_components.md
```

Cada documento de contexto poderá conter os seguintes elementos:

1. Nome da vulnerabilidade;
2. Categoria OWASP relacionada;
3. CWE associada, quando aplicável;
4. Descrição da vulnerabilidade;
5. Causa comum da falha;

6. Impacto de segurança;
7. Recomendações de correção;
8. Cuidados específicos em JavaScript, TypeScript, Node.js, Express ou Angular, quando aplicável;
9. Exemplo de código vulnerável;
10. Exemplo de código corrigido.

Durante a execução do *pipeline*, a ferramenta SAST identificará uma *issue* de segurança. Em seguida, o *pipeline* fará a associação dessa *issue* com uma categoria conhecida de vulnerabilidade. A partir dessa associação, será selecionado automaticamente o documento de contexto correspondente.

Por exemplo, caso a *issue* esteja relacionada a *SQL Injection*, o *pipeline* recuperará o arquivo `sql_injection.md`. Esse conteúdo será então incorporado ao prompt enviado à LLM, juntamente com o código vulnerável e o alerta produzido pela ferramenta SAST.

O fluxo do RAG simplificado será composto pelas seguintes etapas:

1. Identificação da *issue* pela ferramenta SAST;
2. Classificação da *issue* em uma categoria de vulnerabilidade;
3. Seleção automática do arquivo de contexto correspondente;
4. Leitura do conteúdo do arquivo Markdown;
5. Inclusão do contexto recuperado no prompt;
6. Envio do prompt enriquecido para a LLM;
7. Geração da correção.

Essa estratégia permite comparar a correção gerada sem contexto adicional com a correção gerada a partir de um *prompt* enriquecido. Dessa forma, será possível avaliar se a presença de um contexto técnico direcionado melhora a capacidade da LLM de identificar a causa da vulnerabilidade, propor uma correção adequada e preservar a funcionalidade original do sistema.

Embora essa abordagem não implemente todos os elementos de uma arquitetura RAG completa, ela mantém o princípio central da técnica: recuperar conhecimento externo relevante antes da geração da resposta. Por isso, neste trabalho, a estratégia será denominada RAG simplificado ou contextualização baseada em recuperação por categoria.

3.8 Pipeline

O *pipeline* será implementado por meio de scripts responsáveis por orquestrar todas as etapas do experimento. A execução automatizada terá como objetivo reduzir intervenção manual, padronizar a coleta de dados e permitir a repetição dos experimentos.

O fluxo do pipeline será composto pelas seguintes etapas:

1. Executar a análise SAST inicial no projeto;
2. Filtrar vulnerabilidades de segurança relevantes para o experimento;
3. Coletar o trecho de código associado à vulnerabilidades;
4. Classificar a vulnerabilidades de acordo com a categoria de vulnerabilidade;
5. Recuperar o contexto correspondente, quando a estratégia testada utilizar RAG simplificado;
6. Montar o prompt de acordo com a estratégia experimental selecionada;
7. Enviar a requisição à API da LLM;
8. Receber e validar a resposta estruturada;
9. Aplicar a correção proposta em uma cópia isolada do projeto;
10. Executar novamente a análise SAST;
11. Executar a suíte de testes automatizados;
12. Registrar os resultados da tentativa de correção;
13. Restaurar o estado do projeto para a próxima execução.

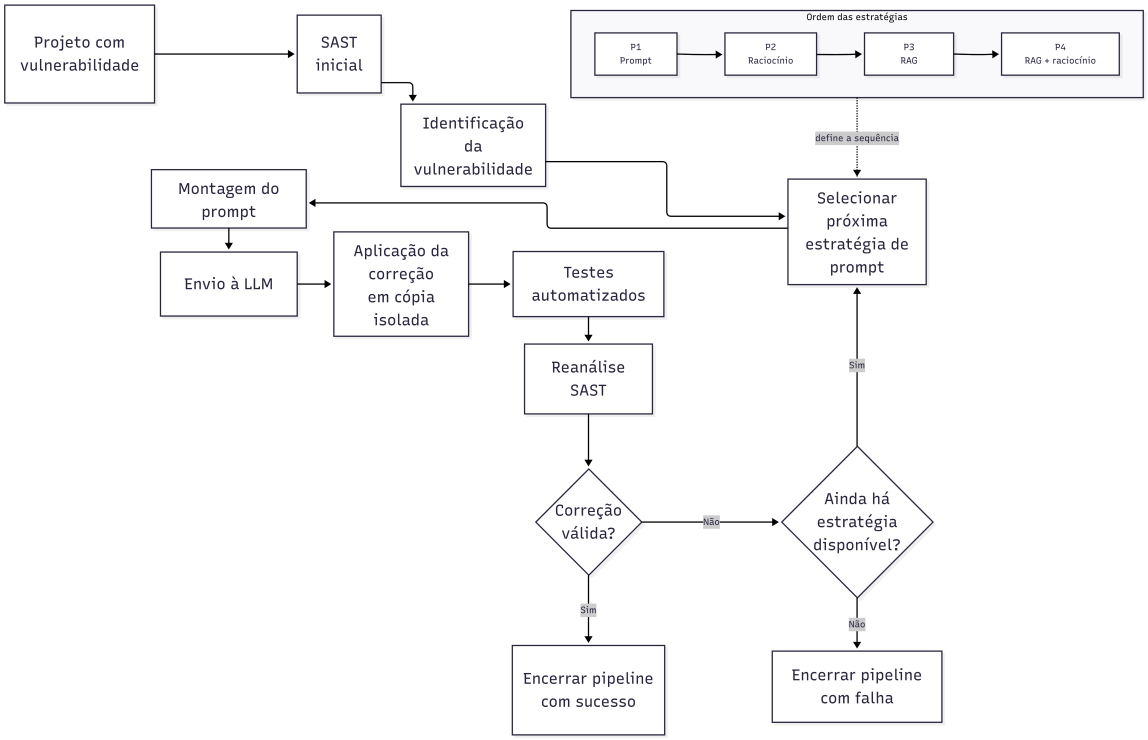


Figura 2 – Fluxo de correção de uma única vulnerabilidade, executado em paralelo para diferentes vulnerabilidades e com tentativas progressivas por estratégias de prompt.

Legenda: P1 = prompt simples; P2 = prompt com raciocínio guiado; P3 = prompt com RAG simplificado; P4 = prompt com RAG simplificado combinado ao raciocínio guiado.

A Figura 2 representa o processamento de uma única vulnerabilidade por instância do pipeline. Após a análise SAST inicial identificar as vulnerabilidades elegíveis no projeto, uma instância independente desse fluxo é criada para cada uma delas. Essas instâncias são executadas em paralelo sobre cópias isoladas do projeto, evitando que as alterações realizadas durante a correção de uma vulnerabilidade interfiram nas demais.

Em cada instância, o pipeline seleciona uma estratégia de prompt, seguindo a ordem P1, P2, P3 e P4. Para cada tentativa, o prompt é montado, enviado à LLM e a correção gerada é aplicada ao código. Após essa etapa, são executados os testes automatizados e uma nova análise SAST. A correção é considerada válida apenas quando a vulnerabilidade deixa de ser detectada, os testes continuam passando e nenhuma nova issue de segurança é introduzida. Caso esses critérios não sejam atendidos, o pipeline avança para a próxima estratégia de prompt; se todas as estratégias forem esgotadas, a tentativa é registrada como falha.

Para reduzir o tempo total de execução, as chamadas à API da LLM associadas a vulnerabilidades distintas poderão ser realizadas em paralelo. Essa paralelização será aplicada de forma controlada, respeitando os limites de taxa da API e garantindo que cada instância opere em uma cópia isolada do projeto ou em um ambiente temporário

independente. Dentro de uma mesma instância, entretanto, as estratégias de prompt permanecerão sequenciais, pois a próxima estratégia somente será utilizada quando a anterior não produzir uma correção válida.

3.9 Métricas de avaliação

As métricas foram definidas para avaliar as tentativas de correção sob três perspectivas: remoção do alerta de segurança, preservação funcional e não introdução de novas vulnerabilidades. Como o estudo será conduzido sobre uma aplicação deliberadamente vulnerável, haverá referências conhecidas sobre parte das falhas existentes. Entretanto, a avaliação do *pipeline* será baseada nos alertas efetivamente identificados pela ferramenta SAST, na reanálise após a correção, nos testes automatizados e, quando necessário, na inspeção manual.

Para cada tentativa de correção submetida ao *pipeline*, serão definidas três variáveis binárias:

- R_j: resultado da reanálise estática. Assume valor 1 se a vulnerabilidade original deixou de ser detectada após a correção, e 0 caso contrário.
- F_j: resultado dos testes automatizados. Assume valor 1 se os testes do projeto passaram após a correção, e 0 caso contrário.
- M_j: introdução de novas vulnerabilidades. Assume valor 1 se nenhuma nova *issue* de segurança foi introduzida após a correção, e 0 caso contrário.

A métrica principal será a Taxa de Remediação Validada (TRV), calculada da seguinte forma:

$$\text{TRV} = \frac{\sum_{j=1}^{m}(R_j \times F_j \times M_j)}{m}$$

Nessa fórmula, m representa o número total de tentativas de correção realizadas no projeto. O uso da multiplicação garante que uma tentativa só seja considerada uma remediação validada quando os três critérios forem satisfeitos simultaneamente: a vulnerabilidade original deixou de ser detectada, os testes continuaram passando e nenhuma nova vulnerabilidade de segurança foi introduzida.

Também serão calculadas métricas complementares. A Taxa de Remoção de *Issues* (TRI) indicará a proporção de tentativas em que a *issue* original deixou de ser detectada:

$$\text{TRI} = \frac{\sum_{j=1}^{m} R_j}{m}$$

A Taxa de Preservação Funcional (TPF) indicará a proporção de tentativas em que os testes automatizados passaram após a correção:

$$\mathrm{TPF} = \frac{\sum_{j=1}^{m} F_j}{m}$$

A Taxa de Não Introdução de Novas *Issues* (TNINI) indicará a proporção de tentativas em que nenhuma nova *issue* de segurança foi introduzida:

$$\mathrm{TNINI} = \frac{\sum_{j=1}^{m} M_j}{m}$$

A Taxa de Falha Funcional (TFF) indicará a proporção de tentativas em que a correção comprometeu os testes automatizados:

$$\mathrm{TFF} = 1 - \mathrm{TPF}$$

Além dessas métricas por tentativa, será calculada a Redução Relativa de *Issues* de Segurança (RRIS), considerando apenas as correções aceitas como válidas:

$$\mathrm{RRIS} = \frac{I_{\mathrm{antes}} - I_{\mathrm{depois}}}{I_{\mathrm{antes}}}$$

Nessa fórmula, I_{antes} representa o número de *issues* de segurança detectadas antes da aplicação das correções aceitas, enquanto I_{depois} representa o número de *issues* de segurança detectadas após essas correções. Essa métrica indica a redução percentual de alertas de segurança no projeto.

A RRIS será interpretada com cautela, pois a redução de *issues* na ferramenta SAST não garante, isoladamente, que todas as vulnerabilidades foram corretamente corrigidas. Por esse motivo, ela será analisada em conjunto com a execução dos testes automatizados e, quando necessário, com inspeção manual de uma amostra das correções geradas.

Essas métricas permitirão comparar o desempenho das diferentes estratégias de *prompt* no projeto. Dessa forma, será possível observar se prompts com raciocínio guiado, prompts com RAG simplificado e prompts com RAG simplificado combinado ao raciocínio guiado apresentam maior taxa de remediação validada em comparação com prompts simples.

3.10 Armazenamento e organização dos dados

Todos os resultados serão armazenados em uma estrutura padronizada de diretórios e arquivos. Cada execução do pipeline produzirá um registro contendo os dados da

vulnerabilidade, o prompt utilizado, a resposta da LLM, o código corrigido, os resultados da reanálise SAST e os resultados dos testes automatizados.

A estrutura de armazenamento seguirá o modelo:

```
outputs/
  sast_antes/
  sast_depois/
  prompts/
  respostas_llm/
  codigos_corrigidos/
  testes/
  metricas/
  resultados_consolidados/
```

Cada tentativa de correção receberá um identificador único, utilizado para nomear os artefatos gerados durante a execução do *pipeline*. Esse identificador foi definido de forma padronizada para facilitar a rastreabilidade dos resultados, permitindo identificar, a partir do próprio nome do arquivo, o caso analisado, a categoria do OWASP Top 10, o tipo de vulnerabilidade, a estratégia de *prompt*, o número da execução e o tipo de artefato produzido.

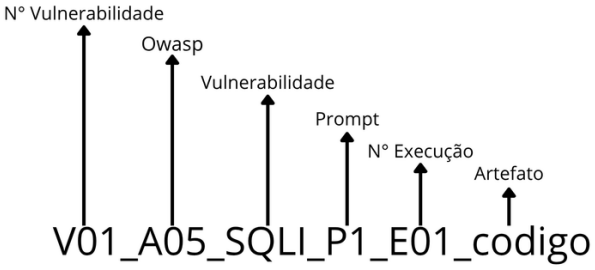


Figura 3 – Estrutura do identificador utilizado na nomenclatura dos artefatos gerados pelo *pipeline*.

Esse identificador permitirá rastrear cada resultado desde o alerta inicial até a correção final. Também permitirá comparar tentativas diferentes para a mesma vulnerabilidade, realizadas com estratégias de *prompt* distintas.

Os resultados consolidados serão armazenados em formato CSV, contendo os campos:

1. identificador da execução;
2. projeto analisado;

3. vulnerabilidade;
4. categoria OWASP;
5. CWE;
6. ferramenta SAST;
7. estratégia de prompt;
8. uso de RAG simplificado;
9. modelo utilizado;
10. *issue* detectada antes da correção;
11. *issue* detectada depois da correção;
12. resultado dos testes;
13. novas *issues* introduzidas;
14. tempo de execução;
15. classificação final da tentativa.

Essa organização garante rastreabilidade e facilita a posterior geração de tabelas, gráficos e análises comparativas.

3.11 Análise dos dados

A análise dos dados será realizada a partir da comparação entre as estratégias de prompt e os tipos de vulnerabilidade. Inicialmente, será calculada a Taxa de Remediação Validada para cada estratégia de *prompt* utilizada.

A análise buscará responder às perguntas de pesquisa definidas no Capítulo 1, observando principalmente:

1. Se o *pipeline* consegue corrigir vulnerabilidades reais identificadas por SAST sem quebrar os testes;
2. Se prompts com raciocínio guiado apresentam melhor desempenho do que prompts simples;
3. Se o uso de RAG simplificado melhora a taxa de remediação validada;

4. Se a combinação entre RAG simplificado e raciocínio guiado apresenta melhor desempenho que as estratégias isoladas;

5. Quais tipos de vulnerabilidades ou regras SAST apresentam maior dificuldade de correção;

6. Quais limitações práticas surgem ao aplicar o *pipeline* em um projeto *open source*.

3.12 Limitações do estudo

Apesar do cuidado metodológico, este estudo apresenta limitações. A primeira delas diz respeito à ferramenta SAST. Ferramentas de análise estática podem gerar falsos positivos e falsos negativos. Nesta pesquisa, não será realizada uma etapa prévia para identificar ou eliminar possíveis falsos positivos. Assim, todas as *issues* detectadas pela ferramenta serão submetidas ao *pipeline* de correção. Consequentemente, o desaparecimento de uma *issue* após a execução do *pipeline* não garante, de forma absoluta, que uma vulnerabilidade real tenha sido corrigida, pois o alerta original pode não corresponder a uma falha efetiva. Por esse motivo, os resultados serão analisados em conjunto com a reanálise SAST, a execução dos testes automatizados e, quando necessário, por meio de inspeção manual.

A segunda limitação está relacionada ao comportamento não determinístico das LLMs. Mesmo utilizando o mesmo prompt, o modelo pode gerar respostas diferentes dependendo dos parâmetros da chamada, da versão do modelo e das condições da API. Para reduzir essa variabilidade, os parâmetros de execução serão registrados e, sempre que possível, mantidos constantes.

A terceira limitação está relacionada ao próprio RAG simplificado. Como a recuperação será feita por categoria de vulnerabilidade, e não por similaridade semântica em uma base vetorial, o método pode recuperar documentos menos específicos em alguns casos. Entretanto, essa escolha é adequada ao escopo do trabalho, pois permite avaliar de forma controlada o efeito do contexto técnico adicional sem transformar a implementação do RAG no foco principal da pesquisa.Ma

A quarta limitação refere-se à utilização de uma única aplicação como objeto de estudo. Embora o OWASP Juice Shop seja uma aplicação amplamente utilizada em pesquisas e treinamentos de segurança, os resultados obtidos refletem o comportamento do *pipeline* nesse contexto específico. Projetos desenvolvidos com outras linguagens, arquiteturas, frameworks ou práticas de desenvolvimento podem apresentar características distintas, influenciando tanto a detecção das *issues* quanto a qualidade das correções geradas pela LLM. Dessa forma, os resultados deste trabalho não devem ser generalizados

para todos os projetos de software, mas interpretados como uma avaliação da abordagem proposta em um cenário representativo.

3.13 Declaração de Uso de Inteligência Artificial

Durante a elaboração deste trabalho, foram utilizadas ferramentas de inteligência artificial generativa como apoio em etapas do processo de escrita e organização do conteúdo. Essas ferramentas auxiliaram, principalmente, na revisão de trechos do texto, na melhoria da clareza de determinadas explicações, na estruturação de ideias, na elaboração de exemplos de código e na organização de diagramas e tabelas.

O uso dessas ferramentas não substituiu o trabalho dos autores. As decisões sobre o tema, a metodologia, os experimentos, as interpretações e a versão final do texto foram realizadas pelos autores. Todo conteúdo gerado com auxílio de inteligência artificial foi revisado, ajustado e validado antes de ser incorporado ao trabalho.

Dessa forma, a inteligência artificial foi utilizada apenas como ferramenta de apoio, enquanto a responsabilidade pelo conteúdo final, pelas escolhas metodológicas e pelas conclusões apresentadas são de responsabilidade integral dos autores.

3.14 Cronograma

As principais atividades previstas para o desenvolvimento do TCC 2 são apresentadas na Tabela 1. O cronograma contempla desde a preparação do ambiente experimental utilizando o OWASP Juice Shop até a análise dos resultados e a redação final do trabalho.

Tabela 1 – Cronograma previsto para o desenvolvimento do TCC 2

Atividade	**Período previsto**
Preparação do ambiente experimental com o OWASP Juice Shop	Agosto de 2026
Estudo do código-fonte e identificação das vulnerabilidades elegíveis	Agosto de 2026
Execução inicial do SAST e catalogação das *issues*	Agosto – Setembro de 2026
Implementação e ajustes do *pipeline* automatizado	Setembro de 2026
Implementação do RAG simplificado	Setembro de 2026
Paralelização das chamadas à API da LLM	Setembro – Outubro de 2026
Definição e refinamento das estratégias de *prompt*	Outubro de 2026
Execução do *pipeline* sobre as vulnerabilidades selecionadas	Outubro – Novembro de 2026
Coleta e consolidação das métricas experimentais	Novembro de 2026
Análise dos resultados e discussão dos achados	Novembro – Dezembro de 2026
Escrita, revisão e preparação da versão final do TCC	Dezembro de 2026

4 Prova de Conceito

4.1 Visão geral

A prova de conceito desenvolvida neste trabalho tem como objetivo avaliar um fluxo automatizado de apoio à correção de vulnerabilidades em código-fonte Python, a partir da integração entre análise estática de segurança, modelos de linguagem de grande escala e testes automatizados.

O experimento foi concebido para simular um processo controlado de identificação, correção e validação de vulnerabilidades. Para isso, foram definidos códigos vulneráveis intencionais, testes automatizados capazes de verificar tanto o comportamento inseguro quanto o comportamento esperado após a correção, estratégias de *prompt* para interação com a LLM e um *pipeline* responsável por orquestrar a execução do fluxo experimental.

A PoC não tem como finalidade representar uma aplicação completa em ambiente de produção. Seu propósito é construir um ambiente experimental controlado, no qual seja possível avaliar se a LLM é capaz de gerar correções úteis para vulnerabilidades previamente selecionadas. Dessa forma, os trechos de código foram mantidos pequenos, isolados e orientados a casos específicos, permitindo observar de maneira mais clara os efeitos das diferentes estratégias de *prompt* sobre a qualidade das correções geradas.

A PoC representa uma etapa preliminar e controlada do estudo. Enquanto o TCC 2 pretende aplicar o *pipeline* em uma base de código mais completa, como o OWASP Juice Shop, a PoC do TCC 1 utiliza uma amostra reduzida, composta por casos vulneráveis criados especificamente para o experimento. Essa separação permite validar, em ambiente controlado, a integração entre análise estática, LLM, testes automatizados, geração de artefatos e cálculo de métricas antes da aplicação em um projeto realista, com maior complexidade estrutural, múltiplos arquivos, dependências externas e suíte de testes própria.

Além da geração das correções, o *pipeline* também realiza a análise estática antes e depois da modificação, executa os testes automatizados e calcula métricas individuais e agregadas para avaliar cada tentativa de correção.

4.2 Elaboração da PoC

A elaboração da PoC foi organizada em cinco etapas principais: seleção das vulnerabilidades, criação dos códigos vulneráveis, construção dos testes automatizados, elaboração das estratégias de *prompt* e definição do fluxo experimental.

4.2.1 Escolha e fundamentação das vulnerabilidades

A versão final da PoC contempla três casos vulneráveis:

```
V01 - SQL Injection simples
V02 - SQL Injection complexa
V03 - IDOR/BOLA
```

A seleção desses casos teve como objetivo representar diferentes níveis de complexidade na correção automatizada de vulnerabilidades. A V01 corresponde a uma vulnerabilidade técnica simples, cuja correção esperada é direta. A V02 pertence à mesma categoria da V01, porém apresenta maior complexidade por envolver identificadores SQL dinâmicos. A V03, por sua vez, representa uma falha de autorização em nível de objeto, dependente da correta aplicação de regras de negócio.

As vulnerabilidades V01 e V02 foram fundamentadas na CWE-89, associada a falhas de *SQL Injection*, bem como na categoria *Injection* da OWASP Top 10 (OWASP Foundation, 2025d). Como base prática de inspiração, utilizou-se a *PortSwigger Web Security Academy*, que apresenta laboratórios didáticos envolvendo a manipulação de entradas controladas pelo usuário em consultas SQL (PortSwigger, 2026). A partir dessa inspiração, foram criados dois casos próprios em Flask, contextualizados em um domínio acadêmico: a V01, na qual o parâmetro `aluno` é inserido diretamente em uma consulta SQL; e a V02, na qual `tabela`, `campo` e `valor` são recebidos pela requisição e empregados na construção da consulta.

A diferença entre a *SQL Injection* simples e a complexa está no nível de construção da consulta SQL. Na V01, apenas o valor do parâmetro `aluno` é inserido diretamente em uma consulta fixa, tornando a correção mais direta por meio de consulta parametrizada. Na V02, além do valor pesquisado, o código também recebe dinamicamente `tabela` e `campo`, exigindo validação por lista de permissão ou mapeamento de valores permitidos. Assim, a V02 representa um caso mais complexo, pois a correção não depende apenas da parametrização do valor, mas também do controle dos identificadores SQL utilizados na consulta.

A vulnerabilidade V03 foi fundamentada na categoria *Broken Access Control* da OWASP Top 10 e na falha de *Broken Object Level Authorization* descrita pela *OWASP API Security Top 10* (OWASP Foundation, 2025a; MITRE, 2026). Também se relaciona à CWE-639, associada à autorização incorreta baseada em identificadores controlados pelo usuário (OWASP Foundation, 2023). Como base prática de inspiração, utilizou-se a *OWASP API Security Top 10*, que descreve cenários em que usuários manipulam identificadores de objetos para acessar recursos sem autorização adequada. A partir dessa inspiração, foi criado um caso próprio de acesso a documentos acadêmicos, no qual o

sistema autentica o usuário, mas não verifica se ele possui permissão para acessar o documento solicitado.

Essas vulnerabilidades foram selecionadas por permitirem a comparação entre três situações distintas:

```
V01 -> correção técnica simples com consulta parametrizada
V02 -> correção técnica intermediária com identificadores SQL dinâmicos
V03 -> correção de regra de negócio envolvendo autorização por objeto
```

Para a padronização dos artefatos, os casos foram associados às seguintes categorias e siglas:

```
V01 -> A05 -> SQLI
V02 -> A05 -> SQLI
V03 -> A01 -> IDOR
```

Nesse mapeamento, `A05` corresponde à categoria *Injection* do OWASP Top 10, `A01` corresponde à categoria *Broken Access Control*, `SQLI` identifica os casos de *SQL Injection* e `IDOR` identifica o caso de IDOR/BOLA.

Essa organização permite analisar se o desempenho da LLM varia conforme o tipo de correção exigida. Em particular, busca-se observar diferenças entre casos em que a correção depende de um padrão técnico conhecido, como o uso de consultas parametrizadas, e casos em que a solução demanda compreensão adicional de regras de acesso e autorização. Ressalta-se que os casos não foram copiados diretamente das bases utilizadas como inspiração; eles foram construídos especificamente para esta PoC, em um domínio acadêmico próprio, pequeno e controlado.

4.2.2 Criação dos códigos vulneráveis

Os códigos vulneráveis foram criados no contexto de um domínio acadêmico, envolvendo documentos, alunos e vínculos de acesso. Essa escolha buscou evitar exemplos excessivamente genéricos e aproximar os casos de um cenário de aplicação web, sem ampliar desnecessariamente a complexidade do código ou dos testes.

Na V01, a vulnerabilidade foi construída de forma propositalmente simples. O parâmetro `aluno` é recebido por uma rota Flask e inserido diretamente em uma consulta SQL:

```
query = f"SELECT * FROM documentos_tcc WHERE aluno = '{aluno}'"
cursor.execute(query)
```

A correção esperada consiste na substituição da interpolação direta por uma consulta parametrizada, pois, nesse formato, a entrada do usuário é tratada como dado e não como parte do comando SQL, impedindo que valores maliciosos alterem a estrutura da consulta (Python Software Foundation, 2026; OWASP Foundation, 2026b). Dessa forma, a saída esperada seria:

```
query = "SELECT * FROM documentos_tcc WHERE aluno = ?"
cursor.execute(query, (aluno,))
```

Na V02, a vulnerabilidade foi criada em uma versão mais complexa. Nesse caso, o código permite que `tabela`, `campo` e `valor` sejam utilizados na construção da consulta SQL. Esse caso foi incluído para avaliar um cenário em que a correção precisa lidar com identificadores SQL dinâmicos. Como nomes de tabelas e colunas não podem ser parametrizados com `?`, a correção esperada deve restringir esses elementos por meio de *allowlist* ou por um mapeamento de consultas fixas permitidas.

Na V03, a vulnerabilidade foi criada em uma rota de visualização de documentos. O sistema identifica o usuário a partir de um cabeçalho da requisição, mas não valida se o usuário autenticado possui vínculo com o documento acessado. Assim, a correção esperada consiste na inclusão de uma verificação de autorização antes do retorno do documento.

4.2.3 Elaboração dos testes automatizados

Após a definição dos códigos vulneráveis, foram elaborados testes automatizados para cada caso. Esses testes possuem duas finalidades principais: verificar se a vulnerabilidade foi efetivamente corrigida e garantir que o comportamento funcional esperado da aplicação não foi comprometido pela correção gerada.

Na V01, os testes validam se uma busca legítima por aluno retorna apenas o documento esperado e se um *payload* como `' OR '1'='1` não retorna todos os registros. Além disso, verificam se o código corrigido deixou de construir consultas SQL por meio de *f-string*, `.format()` ou interpolação insegura.

Na V02, os testes verificam a busca legítima, a rejeição de tabelas inválidas, a rejeição de campos inválidos e a neutralização de *payloads* de *SQL Injection*. Esses testes são relevantes porque uma correção que apenas elimine o alerta da ferramenta SAST pode, ainda assim, comprometer o contrato funcional esperado da função `buscar_registros(tabela, campo, valor)`.

Na V03, os testes verificam se usuários autenticados, mas sem vínculo com determinado documento, são impedidos de acessá-lo. Também validam que usuários autorizados continuam conseguindo acessar os documentos aos quais possuem permissão. Esse caso evidencia a importância dos testes automatizados para vulnerabilidades de autorização,

uma vez que ferramentas SAST podem não identificar adequadamente falhas dependentes de regra de negócio.

4.2.4 Elaboração das estratégias de prompt

Após a definição das vulnerabilidades e dos testes, foram elaboradas quatro estratégias de *prompt* com o objetivo de avaliar se diferentes formas de apresentação do problema à LLM influenciam a qualidade das correções geradas:

```
P1_simples
P2_raciocinio_guiado
P3_rag
P4_rag_raciocinio_guiado
```

Nos artefatos gerados pelo *pipeline*, essas estratégias foram representadas por identificadores reduzidos:

```
P1_simples                -> P1
P2_raciocinio_guiado      -> P2
P3_rag                    -> P3
P4_rag_raciocinio_guiado  -> P4
```

O `P1_simples` corresponde à estratégia mais direta. Nessa abordagem, o código vulnerável é enviado à LLM acompanhado de uma instrução objetiva para identificar a vulnerabilidade, explicar o problema, gerar uma versão corrigida do código e justificar a correção. Esse *prompt* funciona como linha de base do experimento, permitindo comparar seu desempenho com as estratégias que fornecem maior estrutura ou contexto.

O `P2_raciocinio_guiado` foi elaborado como uma forma estruturada de operacionalizar princípios associados ao *Chain-of-Thought*. Em vez de solicitar apenas uma resposta direta, o *prompt* orienta a LLM a decompor a análise em etapas explícitas: identificar a vulnerabilidade, explicar sua causa, descrever o impacto de segurança, propor uma correção no código e justificar tecnicamente por que a solução remove ou mitiga a vulnerabilidade. Essa abordagem busca induzir uma análise intermediária organizada, sem depender da exposição livre ou completa da cadeia de pensamento interna do modelo.

O `P3_rag` combina o diagnóstico produzido pela ferramenta SAST com um contexto textual específico da vulnerabilidade. A *issue* identificada pelo Bandit é simplificada e inserida no *prompt* com dados como tipo da vulnerabilidade, severidade, arquivo, linha, mensagem e regra associada. Além disso, o *prompt* recebe um contexto armazenado em arquivos Markdown no diretório `contextos/`, contendo a descrição da falha, seu impacto,

a base conceitual associada, a correção esperada e os critérios de validação. Essa estratégia foi elaborada para simular um fluxo de RAG, sem utilizar indexação vetorial ou busca semântica. Em vez disso, o contexto relevante é previamente selecionado e anexado diretamente ao *prompt*. Dessa forma, a PoC avalia se o fornecimento de conhecimento adicional, específico e direcionado sobre a vulnerabilidade, combinado ao diagnóstico SAST, contribui para a geração de correções mais adequadas.

O `P4_rag_raciocinio_guiado` combina a estratégia de RAG com raciocínio guiado. Assim como no `P3_rag`, o *prompt* recebe tanto o contexto textual específico da vulnerabilidade quanto as informações da *issue* SAST identificada pelo Bandit. A diferença é que, além desses elementos, a LLM é orientada a seguir uma sequência explícita de análise: identificar a vulnerabilidade com base no código, no contexto recuperado e no achado SAST; explicar a causa da falha; descrever o impacto de segurança; gerar uma versão corrigida do código; justificar a correção; e informar possíveis cuidados adicionais. Essa estratégia busca avaliar se a combinação entre conhecimento contextual, diagnóstico de análise estática e estruturação do raciocínio melhora a qualidade das correções geradas.

Assim, as quatro estratégias permitem comparar diferentes níveis de apoio fornecidos à LLM: instrução direta, raciocínio estruturado, diagnóstico SAST com contexto especializado e diagnóstico SAST com contexto especializado combinado ao raciocínio guiado.

4.2.5 Definição do fluxo experimental

Com as vulnerabilidades, os testes e as estratégias de *prompt* definidos, foi estabelecido o fluxo experimental da PoC. Esse fluxo combina a análise estática realizada pelo Bandit, a geração de correções pela LLM, a validação por meio de testes automatizados e o cálculo das métricas de avaliação.

A execução geral foi definida da seguinte forma:

```
Código vulnerável Python
-> Análise SAST com Bandit antes da correção
-> Construção dos prompts
-> Geração da correção pela LLM
-> Salvamento do código corrigido
-> Análise SAST com Bandit depois da correção
-> Execução dos testes automatizados
-> Consolidação dos resultados por tentativa
-> Cálculo das métricas individuais e agregadas
-> Geração dos arquivos JSON e CSV finais
```

Inicialmente, foi avaliado o uso do SonarQube Community como ferramenta SAST principal. Entretanto, durante os testes preliminares, observou-se que ele não identificou de forma consistente as vulnerabilidades Python utilizadas na PoC. Considerando que o experimento é composto exclusivamente por código Python, optou-se por utilizar o Bandit como ferramenta SAST principal. Essa substituição tornou o fluxo mais aderente ao escopo da PoC, uma vez que o Bandit é especializado em análise estática de segurança para Python e gera relatórios em formato JSON, o que facilita sua integração ao *pipeline*. (Bandit Developers, 2026)

4.3 Estrutura do projeto

A implementação da PoC está disponível em repositório público no GitHub (BRANT; LAMEGO, 2026). A disponibilização do código permite consultar os scripts utilizados, os casos vulneráveis, os testes automatizados e os artefatos produzidos durante a execução do *pipeline*.

A estrutura principal do projeto foi organizada da seguinte forma:

```
PoC-TCC-Matheus-e-Caio/

vulneraveis/
V01_sql_injection.py
V02_sql_injection_complexa.py
V03_idor_bola.py

contextos/
V01_sql_injection_contexto.md
V02_sql_injection_complexa_contexto.md
V03_idor_bola_contexto.md

testes/
test_V01_sql_injection.py
test_V02_sql_injection_complexa.py
test_V03_idor_bola.py

scripts/
artifact_naming.py
bandit_client.py
calculate_metrics.py
openai_client.py
```

```
prompt_builder.py
run_pipeline.py
run_tests.py
summarize_results.py

outputs/
bandit_antes/
bandit_depois/
codigos_corrigidos/
testes/
resultados/
```

O diretório `vulneraveis/` armazena os códigos originais com vulnerabilidades intencionais. Esses arquivos não são sobrescritos durante a execução da PoC, funcionando como entradas fixas do experimento.

O diretório `contextos/` contém os arquivos de contexto utilizados nos *prompts* com contextualização. Cada vulnerabilidade possui um arquivo próprio, contendo descrição da falha, impacto, base de referência, correção esperada e critérios de validação. Esses arquivos não implementam um RAG vetorial, mas funcionam como contexto textual reduzido, anexado diretamente ao *prompt*.

O diretório `testes/` contém os testes automatizados de cada caso. Esses testes verificam se a correção gerada pela LLM preserva o comportamento funcional esperado e elimina o comportamento vulnerável.

O diretório `scripts/` contém os scripts responsáveis pela automação da PoC. O arquivo `artifact_naming.py` centraliza os metadados dos casos, o mapeamento dos *prompts* e a geração padronizada dos nomes dos artefatos. O arquivo `bandit_client.py` executa a análise estática com Bandit. O arquivo `calculate_metrics.py` centraliza o cálculo das variáveis R_i, F_i e M_i, além das métricas agregadas. O arquivo `openai_client.py` centraliza a chamada à API da OpenAI. O arquivo `prompt_builder.py` monta os *prompts*. O arquivo `run_pipeline.py` orquestra o fluxo completo. O arquivo `run_tests.py` executa os testes sobre os códigos corrigidos. Por fim, o arquivo `summarize_results.py` lê o arquivo consolidado `execucoes.json`, recalcula os campos derivados e regenera os CSVs e arquivos de métricas.

O diretório `outputs/` armazena os artefatos gerados durante a execução, incluindo relatórios do Bandit, códigos corrigidos, resultados dos testes e arquivos consolidados.

4.4 Fluxo de execução e armazenamento dos resultados

A execução começa com o arquivo `run_pipeline.py`, que percorre os três casos vulneráveis definidos.

Para manter a rastreabilidade dos artefatos gerados, a PoC adotou o padrão de nomenclatura definido na metodologia deste trabalho. Dessa forma, os arquivos produzidos durante a execução do *pipeline* foram nomeados a partir de um identificador composto pelo caso analisado, pela categoria do OWASP Top 10, pelo tipo de vulnerabilidade, pela estratégia de *prompt*, pelo número da execução e pelo tipo de artefato gerado.

O padrão utilizado é:

```
<CASO>*<OWASP>*<VULN>*<PROMPT>*<EXECUCAO>_<ARTEFATO>.<extensao>
```

Para os relatórios SAST anteriores à correção, utiliza-se o marcador `BASE` no campo de *prompt*, pois ainda não há estratégia aplicada. Para os artefatos gerados após a chamada à LLM, utiliza-se o identificador reduzido da estratégia de *prompt*, como `P1`, `P2`, `P3` ou `P4`.

Para cada caso, o Bandit é executado sobre o arquivo original localizado em `vulneraveis/`. O relatório gerado antes da correção é salvo em:

```
outputs/bandit_antes/<CASO>*<OWASP>*<VULN>*BASE*<EXECUCAO>_sast_antes.json
```

Exemplos:

```
outputs/bandit_antes/V01_A05_SQLI_BASE_E01_sast_antes.json
outputs/bandit_antes/V02_A05_SQLI_BASE_E01_sast_antes.json
outputs/bandit_antes/V03_A01_IDOR_BASE_E01_sast_antes.json
```

Em seguida, o *pipeline* executa as quatro estratégias de *prompt*. Para cada resposta da LLM, o campo `codigo_corrigido` é extraído e salvo como um novo arquivo Python em:

```
outputs/codigos_corrigidos/<CASO>*<OWASP>*<VULN>*<PROMPT>*<EXECUCAO>_codigo.py
```

Exemplos:

```
outputs/codigos_corrigidos/V01_A05_SQLI_P1_E01_codigo.py
outputs/codigos_corrigidos/V02_A05_SQLI_P4_E01_codigo.py
outputs/codigos_corrigidos/V03_A01_IDOR_P2_E01_codigo.py
```

Após o salvamento do código corrigido, o Bandit é executado novamente sobre esse arquivo. O relatório posterior à correção é salvo em:

```
outputs/bandit_depois/<CASO>*<OWASP>*<VULN>*<PROMPT>*<EXECUCAO>_sast_depois.json
```

Exemplos:

```
outputs/bandit_depois/V01_A05_SQLI_P1_E01_sast_depois.json
outputs/bandit_depois/V02_A05_SQLI_P4_E01_sast_depois.json
outputs/bandit_depois/V03_A01_IDOR_P2_E01_sast_depois.json
```

Na sequência, o *pipeline* executa os testes automatizados correspondentes ao caso analisado. Para isso, o código corrigido é copiado para um diretório temporário com o nome esperado pelo teste, como:

```
V01_sql_injection.py
V02_sql_injection_complexa.py
V03_idor_bola.py
```

Essa renomeação temporária é necessária porque os testes importam os módulos pelo nome original do caso. Assim, o código corrigido é testado como se fosse o arquivo vulnerável original, sem alterar os arquivos de entrada da PoC.

O resultado dos testes é salvo em:

```
outputs/testes/<CASO>*<OWASP>*<VULN>*<PROMPT>*<EXECUCAO>_testes.json
```

Exemplos:

```
outputs/testes/V01_A05_SQLI_P1_E01_testes.json
outputs/testes/V02_A05_SQLI_P4_E01_testes.json
outputs/testes/V03_A01_IDOR_P2_E01_testes.json
```

Ao final da execução, os resultados consolidados são salvos em:

```
outputs/resultados/execucoes.json
outputs/resultados/execucoes.csv
outputs/resultados/metricas.json
outputs/resultados/metricas.csv
outputs/resultados/metricas_por_prompt.csv
outputs/resultados/metricas_por_caso.csv
```

O arquivo `execucoes.json` mantém os dados detalhados da execução, incluindo caminhos dos artefatos gerados e contagens de *issues* antes e depois. O arquivo `execucoes.csv` apresenta uma visão tabular das tentativas realizadas. Os arquivos `metricas.json` e `metricas.csv` armazenam as métricas gerais, enquanto `metricas_por_prompt.csv` e `metricas_por_caso.csv` agrupam os resultados por estratégia de *prompt* e por caso vulnerável.

4.5 Métricas de avaliação

As métricas utilizadas na PoC seguem a definição apresentada na metodologia do trabalho. Nesta etapa, elas foram aplicadas aos resultados gerados pelo *pipeline* para classificar cada tentativa de correção e consolidar o desempenho das estratégias de *prompt*.

Para cada tentativa, o *pipeline* registra três variáveis binárias: S_i, T_i e N_i. A variável S_i indica se a *issue* original detectada pelo Bandit deixou de aparecer após a correção. A variável T_i indica se os testes automatizados passaram após a modificação. A variável N_i indica se nenhuma nova *issue* de segurança foi introduzida pelo código corrigido.

A tentativa é classificada como adequada apenas quando os três critérios são satisfeitos ao mesmo tempo. Portanto, uma correção que remove o alerta SAST, mas quebra os testes, não é considerada adequada. Da mesma forma, uma correção que passa nos testes, mas introduz novas *issues*, também não é considerada bem-sucedida.

Para comparar as *issues* antes e depois da correção, o *pipeline* utiliza uma assinatura estável composta pelos campos `test_id`, `issue_text`, `issue_severity` e `issue_confidence` do relatório Bandit. O número da linha não é utilizado como critério principal, pois a posição do código pode ser alterada pela correção gerada pela LLM.

Além da classificação individual de cada tentativa, o *pipeline* calcula métricas agregadas: Taxa de Remediação Validada (TRV), Taxa de Remoção de *Issues* (TRI), Taxa de Preservação Funcional (TPF), Taxa de Não Introdução de Novas *Issues* (TNINI) e Taxa de Falha Funcional (TFF). Esses valores são gerados de forma geral, por estratégia de *prompt* e por caso vulnerável.

No caso da V03, por se tratar de uma vulnerabilidade de regra de negócio, é possível que o Bandit não identifique a falha inicial. Nesses casos, S_i permanece igual a 0, pois não houve *issue* SAST inicial a ser removida. Para permitir a análise funcional da correção, o *pipeline* registra também o campo `avaliacao_por_testes`, que indica se os testes automatizados foram aprovados independentemente da detecção inicial pelo SAST.

4.6 Execução da PoC

A execução é realizada a partir da raiz do projeto.

Primeiro, prepara-se o ambiente:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

O arquivo `.env` deve conter:

```
OPENAI_API_KEY=<chave_da_api>
OPENAI_MODEL=gpt-4.1-mini
BANDIT_CMD=bandit
```

Em seguida, executa-se o *pipeline*:

```
python3 scripts/run_pipeline.py
```

Caso seja necessário gerar novamente os CSVs e as métricas a partir do JSON consolidado, sem executar novamente as chamadas à LLM, utiliza-se:

```
python scripts/summarize_results.py
```

Os principais resultados ficam em:

```
outputs/resultados/execucoes.json
outputs/resultados/execucoes.csv
outputs/resultados/metricas.json
outputs/resultados/metricas.csv
outputs/resultados/metricas_por_prompt.csv
outputs/resultados/metricas_por_caso.csv
```

4.7 Interpretação dos resultados

O arquivo `execucoes.csv` permite comparar o desempenho de cada tentativa de correção. Ele registra, para cada caso e estratégia de *prompt*, se a *issue* foi detectada antes da correção, se deixou de aparecer depois, se os testes passaram, se novas *issues* foram introduzidas e qual foi a classificação final da tentativa.

As Figuras 4 e 5 apresentam um exemplo da execução completa do *pipeline* no terminal, incluindo o processamento dos casos vulneráveis, a aplicação das estratégias de *prompt*, a geração das correções pela LLM, a execução da análise estática e dos testes automatizados, além do salvamento dos resultados e do cálculo das métricas gerais.

```
poc/PoC-TCC-Matheus-e-Caio$ python3 scripts/run_pipeline.py

=== Analisando caso V01 - sql_injection ===
Rodando Bandit antes da correção...

Executando V01 - P1 (P1_simples)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True

Executando V01 - P2 (P2_raciocinio_guiado)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True

Executando V01 - P3 (P3_rag)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True

Executando V01 - P4 (P4_rag_raciocinio_guiado)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True

=== Analisando caso V02 - sql_injection_complexa ===
Rodando Bandit antes da correção...

Executando V02 - P1 (P1_simples)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? False

Executando V02 - P2 (P2_raciocinio_guiado)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? False

Executando V02 - P3 (P3_rag)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True
```

Figura 4 – Primeira parte da execução completa do *pipeline* da PoC.

```
Executando V02 - P4 (P4_rag_raciocinio_guiado)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True

=== Analisando caso V03 - idor_bola ===
Rodando Bandit antes da correção...

Executando V03 - P1 (P1_simples)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? False

Executando V03 - P2 (P2_raciocinio_guiado)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? False

Executando V03 - P3 (P3_rag)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True

Executando V03 - P4 (P4_rag_raciocinio_guiado)
Rodando Bandit depois da correção...
Executando testes automatizados...
Correção adequada? True
Métricas gerais: TCBS=0.6667, TRI=0.5, TPF=0.6667, TNINI=1.0, TFF=0.3333

Resultados salvos em: /home/matheus-brant/Desktop/repoc/PoC-TCC-Matheus-e-Caio/outpu
ts/resultados/execucoes.json
Resumo CSV salvo em: /home/matheus-brant/Desktop/repoc/PoC-TCC-Matheus-e-Caio/output
s/resultados/execucoes.csv
```

Figura 5 – Segunda parte da execução completa do *pipeline* da PoC.

Após a execução do *pipeline*, foram gerados automaticamente os artefatos da PoC, incluindo os códigos corrigidos, os relatórios da análise estática antes e depois das correções, os resultados dos testes automatizados e os arquivos consolidados de métricas. Esses resultados são apresentados nas tabelas a seguir, que sintetizam o comportamento de cada tentativa de correção, as métricas gerais da execução, a comparação entre as estratégias de *prompt* e a análise consolidada por caso vulnerável.

As informações de resultado foram divididas em duas tabelas para melhorar a legibilidade e a adequação ao formato do PDF. A Tabela 2 apresenta os resultados operacionais de cada tentativa de correção, enquanto a Tabela 3 apresenta as métricas individuais e a classificação final das correções. Em conjunto, essas tabelas consolidam os resultados obtidos pelo *pipeline* para todas as combinações entre casos vulneráveis e estratégias de *prompt* executadas na PoC.

Tabela 2 – Resultados operacionais das execuções da PoC

Caso	OWASP	Vuln.	Prompt	Exec.	Issue antes	Issue sumiu	Testes	Novas issues
V01	A05	SQLI	P1	E01	true	true	true	false
V01	A05	SQLI	P2	E01	true	true	true	false
V01	A05	SQLI	P3	E01	true	true	true	false
V01	A05	SQLI	P4	E01	true	true	true	false
V02	A05	SQLI	P1	E01	true	false	false	false
V02	A05	SQLI	P2	E01	true	false	false	false
V02	A05	SQLI	P3	E01	true	true	true	false
V02	A05	SQLI	P4	E01	true	true	true	false
V03	A01	IDOR	P1	E01	false	false	false	false
V03	A01	IDOR	P2	E01	false	false	false	false
V03	A01	IDOR	P3	E01	false	false	true	false
V03	A01	IDOR	P4	E01	false	false	true	false

Tabela 3 – Métricas e classificação final das correções

Caso	OWASP	Vuln.	Prompt	Exec.	R_i	F_i	M_i	Correção adequada	Avaliação por testes
V01	A05	SQLI	P1	E01	1	1	1	true	true
V01	A05	SQLI	P2	E01	1	1	1	true	true
V01	A05	SQLI	P3	E01	1	1	1	true	true
V01	A05	SQLI	P4	E01	1	1	1	true	true
V02	A05	SQLI	P1	E01	0	0	1	false	false
V02	A05	SQLI	P2	E01	0	0	1	false	false
V02	A05	SQLI	P3	E01	1	1	1	true	true
V02	A05	SQLI	P4	E01	1	1	1	true	true
V03	A01	IDOR	P1	E01	0	0	1	false	false
V03	A01	IDOR	P2	E01	0	0	1	false	false
V03	A01	IDOR	P3	E01	0	1	1	true	true
V03	A01	IDOR	P4	E01	0	1	1	true	true

Com isso, observa-se que a V01 apresentou correções adequadas nas quatro estratégias de *prompt*, pois a *issue* original deixou de aparecer, os testes automatizados foram aprovados e nenhuma nova *issue* foi introduzida. A V02 apresentou maior dificuldade, uma vez que apenas duas estratégias conseguiram satisfazer simultaneamente os critérios de segurança e funcionalidade. Já a V03 evidencia uma limitação da análise estática: por se tratar de uma vulnerabilidade relacionada a controle de acesso e regra de negócio, o Bandit não detectou uma *issue* inicial a ser removida. Ainda assim, nas estratégias em que os testes foram aprovados, a correção foi considerada adequada pela avaliação complementar baseada em testes, demonstrando a importância de combinar análise estática com validação funcional e de segurança.

Tabela 4 – Métricas gerais obtidas na execução da PoC

Total de tentativas	TRV	TRI	TPF	TNINI	TFF	Correções adequadas
12	0.6667	0.5	0.6667	1.0	0.3333	8

A Tabela 4 apresenta as métricas gerais calculadas a partir do conjunto completo de resultados produzidos em todas as execuções da PoC. Esses valores consolidam o desempenho global do *pipeline*, considerando as 12 tentativas realizadas a partir da combinação entre os três casos vulneráveis e as quatro estratégias de *prompt*. Dessa forma, a tabela permite observar, de maneira agregada, a taxa de correções adequadas, a remoção

de *issues*, a preservação funcional, a não introdução de novas *issues* e a ocorrência de falhas funcionais.

Tabela 5 – Métricas consolidadas por estratégia de *prompt*

Prompt	Total	TRV	TRI	TPF	TNINI	TFF	Correções adequadas
P1	3	0.3333	0.3333	0.3333	1.0	0.6667	1
P2	3	0.3333	0.3333	0.3333	1.0	0.6667	1
P3	3	1.0	0.6667	1.0	1.0	0.0	3
P4	3	1.0	0.6667	1.0	1.0	0.0	3

A Tabela 5 apresenta as métricas consolidadas por estratégia de *prompt*, permitindo comparar o desempenho das abordagens `P1`, `P2`, `P3` e `P4`. Observa-se que as estratégias `P1` e `P2` obtiveram apenas uma correção adequada entre três tentativas, com TRV de 0,3333 e TFF de 0,6667. Em contraste, as estratégias `P3` e `P4` apresentaram melhor desempenho, com três correções adequadas em três tentativas, TRV igual a 1,0 e ausência de falhas funcionais. Esse resultado sugere que a inclusão de informações adicionais no *prompt*, como contexto específico da vulnerabilidade e dados provenientes da análise SAST, contribuiu para a geração de correções mais adequadas nos casos avaliados.

Tabela 6 – Métricas consolidadas por caso vulnerável

Caso	OWASP	Vuln.	Total	TRV	TRI	TPF	TNINI	TFF	Correções adequadas
V01	A05	SQLI	4	1.0	1.0	1.0	1.0	0.0	4
V02	A05	SQLI	4	0.5	0.5	0.5	1.0	0.5	2
V03	A01	IDOR	4	0.5	0.0	0.5	1.0	0.5	2

A Tabela 6 apresenta as métricas consolidadas por caso vulnerável, permitindo observar diferenças de desempenho entre V01, V02 e V03. Os resultados indicam que a V01 foi o caso com melhor desempenho, alcançando quatro correções adequadas em quatro tentativas, com TRV, TRI e TPF iguais a 1,0. A V02 apresentou maior complexidade, com duas correções adequadas em quatro tentativas, refletindo a dificuldade adicional de corrigir consultas SQL com identificadores dinâmicos. Já a V03 também obteve duas correções adequadas em quatro tentativas, mas apresentou TRI igual a 0,0, pois o Bandit não detectou uma *issue* inicial relacionada à falha de IDOR/BOLA. Esse resultado evidencia que vulnerabilidades dependentes de regra de negócio exigem validação complementar por testes automatizados.

Essa interpretação reforça a importância da combinação entre análise estática e testes automatizados. A análise estática contribui para identificar padrões inseguros no código, enquanto os testes verificam se a correção preserva o comportamento esperado e bloqueia o comportamento vulnerável. Além disso, as métricas agregadas permitem comparar as estratégias de *prompt* e identificar quais casos apresentaram maior dificuldade de correção.

4.8 Contribuição da PoC

A principal contribuição da PoC é demonstrar um fluxo automatizado de correção e validação de vulnerabilidades em código Python. O *pipeline* permite comparar diferentes estratégias de *prompt*, registrar os códigos corrigidos gerados pela LLM e avaliar os resultados por meio de análise estática, testes automatizados e métricas consolidadas.

Com essa organização, o experimento não se limita a verificar se a LLM produz uma resposta aparentemente correta. Ele avalia se a correção remove o alerta de segurança, preserva o comportamento funcional e não introduz novas *issues*. Além disso, a PoC padroniza a nomenclatura dos artefatos, preserva a rastreabilidade entre código corrigido, relatórios SAST e testes, calcula métricas por tentativa e gera resultados consolidados de forma geral, por estratégia de *prompt* e por caso vulnerável.

A PoC também evidencia uma limitação importante das ferramentas SAST: vulnerabilidades dependentes de regra de negócio, como IDOR/BOLA, podem não ser detectadas pela análise estática. Nesses casos, os testes automatizados assumem papel central na avaliação da correção, demonstrando a importância de combinar diferentes mecanismos de validação em fluxos automatizados de apoio à segurança de software.

5 Considerações Finais

Este trabalho teve como objetivo investigar a possibilidade de integrar Modelos de Linguagem de Grande Escala, ferramentas de Análise Estática de Segurança e estratégias de contextualização para apoiar a correção automatizada de vulnerabilidades em código-fonte. A motivação central da pesquisa parte do fato de que LLMs podem auxiliar na geração de correções, mas ainda não oferecem garantias suficientes quando utilizadas de forma isolada. Da mesma forma, ferramentas SAST são úteis para identificar possíveis falhas, mas seus alertas precisam ser interpretados, validados e tratados de maneira adequada.

Ao longo do TCC 1, foi construída a fundamentação teórica necessária para sustentar a proposta, abordando LLMs, vulnerabilidades de software, OWASP Top 10, engenharia de prompt, RAG, *Chain-of-Thought* e ferramentas SAST. Essa base permitiu delimitar o problema de pesquisa e orientar a construção de um pipeline experimental voltado à correção de *issues* de segurança identificadas por análise estática.

A metodologia definida para o TCC 2 passou a se concentrar na aplicação do pipeline em um projeto *open source*. Essa decisão torna o estudo mais próximo de um cenário prático de desenvolvimento, pois as vulnerabilidades analisadas não serão inseridas manualmente, mas identificadas a partir dos alertas emitidos pela ferramenta SAST no próprio projeto selecionado. Dessa forma, o trabalho deixa de depender de um conjunto previamente controlado de falhas e passa a avaliar o comportamento da abordagem em uma base de código real, com estrutura, dependências, testes e limitações próprias.

A prova de conceito desenvolvida nesta etapa teve papel importante na validação inicial da proposta. Ela demonstrou a viabilidade técnica de automatizar a execução de uma ferramenta SAST, coletar diagnósticos, montar diferentes estratégias de *prompt*, enviar requisições à LLM, aplicar correções geradas, executar testes automatizados e consolidar métricas de avaliação. Embora tenha sido realizada com casos reduzidos e controlados, a PoC permitiu observar aspectos relevantes para a continuidade do trabalho, como a importância da rastreabilidade dos artefatos, da padronização das respostas da LLM e da combinação entre reanálise estática e testes funcionais.

Assim, o TCC 1 estabelece as bases para a continuidade da pesquisa. A principal contribuição desta etapa está na definição de um fluxo estruturado para avaliar correções geradas por LLMs de forma mais rigorosa, evitando falsas correções. A continuidade do trabalho permitirá verificar até que ponto a integração entre SAST, LLMs e estratégias de contextualização pode contribuir para apoiar a correção de vulnerabilidades de maneira segura, rastreável e funcional.

Referências

Bandit Developers. *Welcome to Bandit*. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://bandit.readthedocs.io/>. Citado na página 56.

BRANT, M. d. S.; LAMEGO, C. M. *PoC-TCC-Matheus-e-Caio*. 2026. GitHub. Disponível em: <https://github.com/MatheussBrant/PoC-TCC-Matheus-e-Caio>. Acesso em: 20 jun. 2026. Disponível em: <https://github.com/MatheussBrant/PoC-TCC-Matheus-e-Caio>. Citado na página 56.

CHEN, M. et al. *Evaluating Large Language Models Trained on Code*. arXiv, 2021. ArXiv:2107.03374 [cs.LG]. Disponível em: <http://arxiv.org/abs/2107.03374>. Citado na página 20.

CVE Program. *CVE: Common Vulnerabilities and Exposures*. 2026. Acesso em: 09 jun. 2026. Disponível em: <https://www.cve.org/about/overview>. Citado na página 26.

DAI, S.-C.; XU, J.; TAO, G. *Rethinking the Evaluation of Secure Code Generation*. arXiv, 2025. ArXiv:2503.15554 [cs]. Disponível em: <http://arxiv.org/abs/2503.15554>. Citado 3 vezes nas páginas 13, 20 e 31.

Forum of Incident Response and Security Teams. *Common Vulnerability Scoring System version 4.0: Specification Document*. 2023. Acesso em: 09 jun. 2026. Disponível em: <https://www.first.org/cvss/specification-document>. Citado na página 26.

IRANMANESH, M. et al. *ZeroFalse: Improving Precision in Static Analysis with LLMs*. arXiv, 2025. ArXiv:2510.02534 [cs]. Disponível em: <http://arxiv.org/abs/2510.02534>. Citado 5 vezes nas páginas 12, 13, 14, 29 e 30.

JIANG, N. et al. Impact of Code Language Models on Automated Program Repair. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. Melbourne, Australia: IEEE, 2023. p. 1430–1442. ISBN 978-1-6654-5701-9. Disponível em: <https://ieeexplore.ieee.org/document/10172517/>. Citado na página 18.

KHAN, A.; LIU, G.; GAO, X. Code Vulnerability Repair with Large Language Model using Context-Aware Prompt Tuning. In: *2025 IEEE Security and Privacy Workshops (SPW)*. [s.n.], 2025. p. 283–287. ArXiv:2409.18395 [cs]. Disponível em: <http://arxiv.org/abs/2409.18395>. Citado 5 vezes nas páginas 13, 21, 22, 24 e 27.

KLESEL, M.; WITTMANN, H. F. Retrieval-Augmented Generation (RAG). *Business & Information Systems Engineering*, v. 67, n. 4, p. 551–561, ago. 2025. ISSN 1867-0202. Disponível em: <https://doi.org/10.1007/s12599-025-00945-3>. Citado 2 vezes nas páginas 23 e 24.

KOJIMA, T. et al. *Large Language Models are Zero-Shot Reasoners*. arXiv, 2023. ArXiv:2205.11916 [cs.CL]. Disponível em: <http://arxiv.org/abs/2205.11916>. Citado na página 22.

LESTER, B.; AL-RFOU, R.; CONSTANT, N. *The Power of Scale for Parameter-Efficient Prompt Tuning*. arXiv, 2021. ArXiv:2104.08691 [cs.CL]. Disponível em: <http://arxiv.org/abs/2104.08691>. Citado 2 vezes nas páginas 21 e 22.

LIU, M. X. et al. "We Need Structured Output": Towards User-centered Constraints on Large Language Model Output. In: *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. [s.n.], 2024. p. 1–9. ArXiv:2404.07362 [cs.HC]. Disponível em: <http://arxiv.org/abs/2404.07362>. Citado na página 21.

LU, Y. et al. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In: MURESAN, S.; NAKOV, P.; VILLAVICENCIO, A. (Ed.). *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Dublin, Ireland: Association for Computational Linguistics, 2022. p. 8086–8098. Disponível em: <https://aclanthology.org/2022.acl-long.556/>. Citado 2 vezes nas páginas 21 e 22.

MINAEE, S. et al. *Large Language Models: A Survey*. arXiv, 2025. ArXiv:2402.06196 [cs.CL]. Disponível em: <http://arxiv.org/abs/2402.06196>. Citado 2 vezes nas páginas 18 e 19.

MITRE. *CWE-639: Authorization Bypass Through User-Controlled Key*. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://cwe.mitre.org/data/definitions/639.html>. Citado na página 51.

MITRE Corporation. *CWE: About the Common Weakness Enumeration*. 2024. Acesso em: 09 jun. 2026. Disponível em: <https://cwe.mitre.org/about/index.html>. Citado na página 26.

National Institute of Standards and Technology. *Source Code Security Analyzers*. 2021. Criado em 23 mar. 2021. Última modificação em 01 maio 2026. Acesso em: 11 maio 2026. Disponível em: <https://www.nist.gov/itl/csd/secure-systems-and-applications/source-code-security-analyzers>. Citado 2 vezes nas páginas 28 e 30.

National Institute of Standards and Technology. *National Vulnerability Database*. 2026. Acesso em: 09 jun. 2026. Disponível em: <https://nvd.nist.gov/>. Citado na página 26.

OPENAI. *GPT-4.1 mini Model*. 2026. OpenAI API Documentation. Disponível em: <https://developers.openai.com/api/docs/models/gpt-4.1-mini>. Acesso em: 20 jun. 2026. Disponível em: <https://developers.openai.com/api/docs/models/gpt-4.1-mini>. Citado na página 35.

OSMANI, A. *Vibe coding is not the same as AI-Assisted engineering*. 2025. Disponível em: <https://medium.com/@addyosmani/vibe-coding-is-not-the-same-as-ai-assisted-engineering-3f81088d5b98>. Citado na página 12.

OWASP Foundation. *API1:2023 - Broken Object Level Authorization*. 2023. Acesso em: 18 jun. 2026. Disponível em: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>. Citado na página 51.

OWASP Foundation. *A01:2025 – Broken Access Control*. 2025. <https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/>. Acesso em: 22 jun. 2026. Citado na página 51.

OWASP Foundation. *A01:2025 - Broken Access Control*. 2025. Acesso em: 18 jun. 2026. Disponível em: <https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/>. Citado na página 27.

OWASP Foundation. *A04:2025 - Cryptographic Failures*. 2025. Acesso em: 18 jun. 2026. Disponível em: <https://owasp.org/Top10/2025/A04_2025-Cryptographic_Failures/>. Citado na página 27.

OWASP Foundation. *A05:2025 - Injection*. 2025. Acesso em: 18 jun. 2026. Disponível em: <https://owasp.org/Top10/2025/A05_2025-Injection/>. Citado 2 vezes nas páginas 27 e 51.

OWASP Foundation. *A07:2025 - Authentication Failures*. 2025. Acesso em: 18 jun. 2026. Disponível em: <https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/>. Citado na página 27.

OWASP Foundation. *A10:2025 - Mishandling of Exceptional Conditions*. 2025. Acesso em: 18 jun. 2026. Disponível em: <https://owasp.org/Top10/2025/A10_2025-Mishandling_of_Exceptional_Conditions/>. Citado na página 27.

OWASP Foundation. *Introduction – OWASP Top 10:2025*. 2025. Acesso em: 11 maio 2026. Disponível em: <https://owasp.org/Top10/2025/0x00_2025-Introduction/>. Citado 3 vezes nas páginas 26, 27 e 28.

OWASP Foundation. *OWASP Juice Shop*. 2026. Projeto oficial do OWASP Juice Shop. Disponível em: <https://owasp.org/www-project-juice-shop/>. Citado na página 34.

OWASP Foundation. *SQL Injection Prevention Cheat Sheet*. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html>. Citado na página 53.

OWASP Foundation. *Static Code Analysis*. 2026. Autor da página: Ryan Dewhurst. Acesso em: 11 maio 2026. Disponível em: <https://owasp.org/www-community/controls/Static_Code_Analysis>. Citado 3 vezes nas páginas 28, 29 e 30.

PEARCE, H. et al. *Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions*. arXiv, 2021. ArXiv:2108.09293 [cs]. Disponível em: <http://arxiv.org/abs/2108.09293>. Citado na página 12.

PortSwigger. *SQL injection*. 2026. <https://portswigger.net/web-security/sql-injection>. Acesso em: 12 jun. 2026. Citado na página 51.

Python Software Foundation. *sqlite3 — DB-API 2.0 interface for SQLite databases*. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://docs.python.org/3/library/sqlite3.html>. Citado na página 53.

REYNOLDS, L.; MCDONELL, K. *Prompt Programming for Large Language Models: Beyond the Few-Shot Paradigm*. arXiv, 2021. ArXiv:2102.07350 [cs.CL]. Disponível em: <http://arxiv.org/abs/2102.07350>. Citado 3 vezes nas páginas 21, 22 e 31.

Similarweb. *Top AI Chatbots and Tools Websites Ranking*. 2026. Acesso em: 18 jun. 2026. Disponível em: <https://www.similarweb.com/top-websites/ai-chatbots-and-tools/>. Citado na página 35.

SonarSource. *SonarQube Rules*. 2026. Documentação do SonarQube Server. Acesso em: 11 maio 2026. Disponível em: <https://docs.sonarsource.com/sonarqube-server/quality-standards-administration/managing-rules/rules>. Citado na página 30.

SRIRAM, V. et al. *Improving LLM-Assisted Secure Code Generation through Retrieval-Augmented-Generation and Multi-Tool Feedback*. arXiv, 2026. ArXiv:2601.00509 [cs]. Disponível em: <http://arxiv.org/abs/2601.00509>. Citado 5 vezes nas páginas 13, 14, 18, 20 e 32.

Stack Overflow. *2025 Developer Survey: AI*. 2025. Acesso em: 18 jun. 2026. Disponível em: <https://survey.stackoverflow.co/2025/ai>. Citado na página 35.

SáNCHEZ, M. C. et al. Software vulnerabilities overview: A descriptive study. *Tsinghua Science and Technology*, TUP, v. 25, n. 2, p. 270–280, abr. 2020. ISSN 1007-0214. Citado na página 25.

TAO, W. et al. Adapting LLM to Generate Vulnerability Descriptions and Repair Suggestions. In: *2025 IEEE 5th International Conference on Software Engineering and Artificial Intelligence (SEAI)*. [s.n.], 2025. p. 184–188. Disponível em: <https://ieeexplore.ieee.org/document/11108759>. Citado 2 vezes nas páginas 14 e 31.

VASWANI, A. et al. *Attention Is All You Need*. arXiv, 2023. ArXiv:1706.03762 [cs.CL]. Disponível em: <http://arxiv.org/abs/1706.03762>. Citado na página 17.

WANG, X. et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. arXiv, 2023. ArXiv:2203.11171 [cs.CL]. Disponível em: <http://arxiv.org/abs/2203.11171>. Citado na página 23.

WEI, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv, 2023. ArXiv:2201.11903 [cs.CL]. Disponível em: <http://arxiv.org/abs/2201.11903>. Citado na página 22.

ZHOU, X. et al. *Large Language Model for Vulnerability Detection and Repair: Literature Review and the Road Ahead*. arXiv, 2024. ArXiv:2404.02525 [cs.SE]. Disponível em: <http://arxiv.org/abs/2404.02525>. Citado 3 vezes nas páginas 12, 18 e 19.